04 - Parzen Windows

Pattern Recognition and Machine Learning, Lab 4

Michal Neoral
neoramic@fel.cvut.cz

16 October 2026

Test first

Test now

- **Bayesian** and **non-Bayesian** tasks
- Calculator only: no phone, laptop or notes
- Write your **name** on every sheet

Recap

What do we know about the classes?

Bayesian task

- $p(x \mid k)$ **known**
- $p(k)$ **known**
- $W(k, d)$ **known**

We know **everything**.

Non-Bayesian tasks

- $p(x \mid k)$ **known**
- $p(k)$ **unknown**
- $W(k, d)$ **unknown**

Neyman-Pearson, minimax, Wald

Today

- $p(x \mid k)$ **unknown**
- only **samples** $x_1, \ldots, x_N$

We have to **estimate** the density.

today

! Once we have the estimate $\hat{p}(x \mid k)$, we plug it into the **Bayesian strategy** as before.

Probability density estimation

Each class is estimated **separately**, from its own samples: we drop k and write $p(x)$ instead of $p(x \mid k)$.

? How can we estimate $p(x)$ from samples?

Parametric

Known **form** (e.g. normal), unknown parameters.

MLE, MAP, Bayesian estimation

next lab

Non-parametric

Any shape of the density.

histogram, k-NN, **Parzen windows** (kernel density estimation)

today

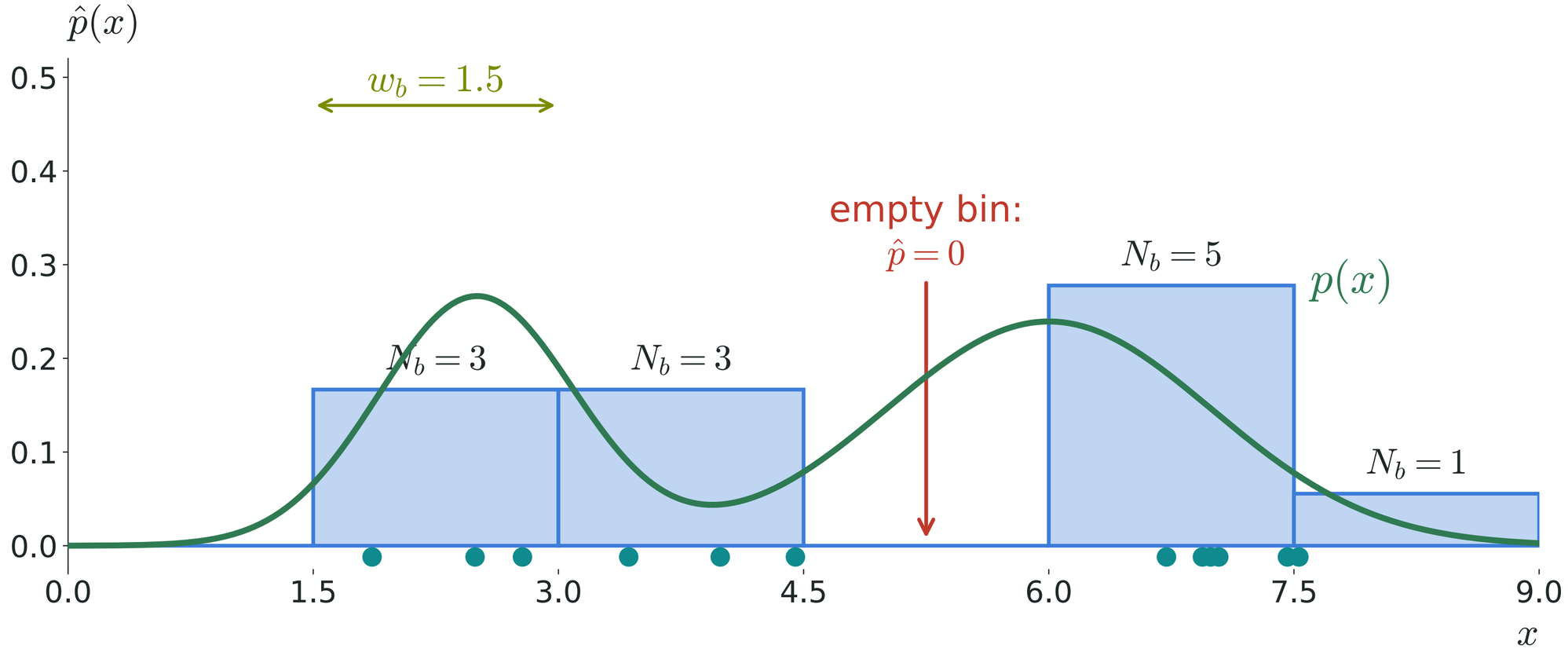
Mixtures

Combine simple densities.

mixture of Gaussians, neural networks

Histogram

The histogram as a density estimate



Compared with the true $p(x)$: one bin in a dense region is **empty**.

Derivation: the height of a bin

The bin contains N_b of the N samples:

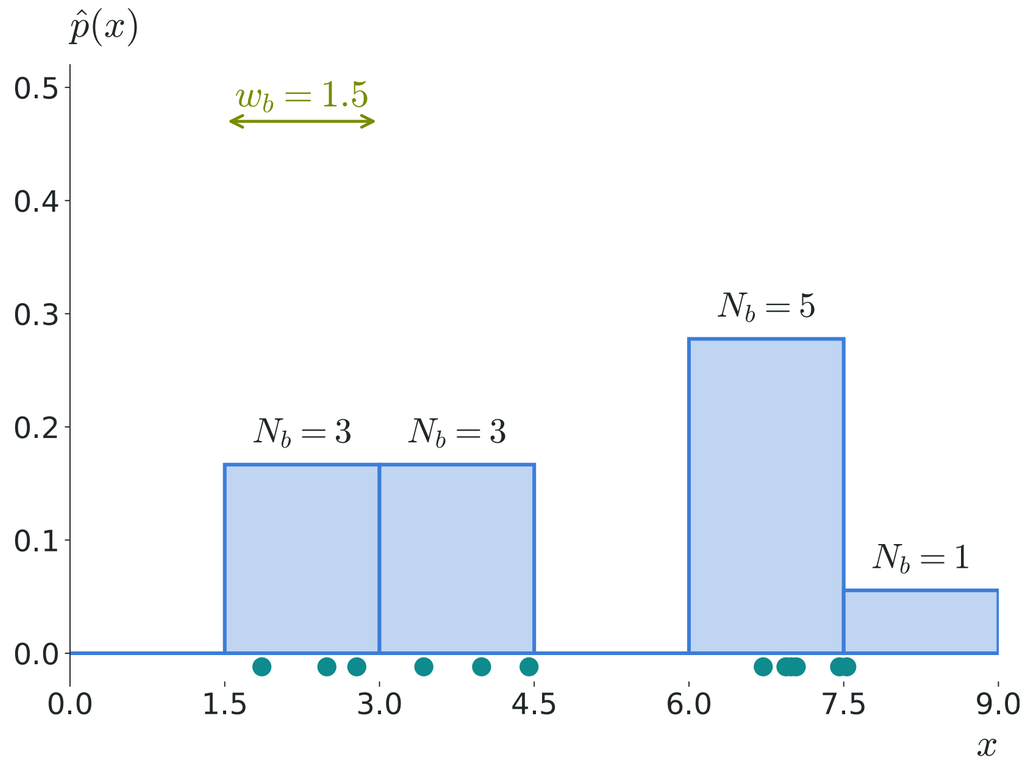
$$P(x \in \text{bin } b) \approx \frac{N_b}{N}$$

Density is probability **per unit length**:

$$\hat{p}_b = \frac{N_b}{N \cdot w_b}$$

The estimate integrates to one:

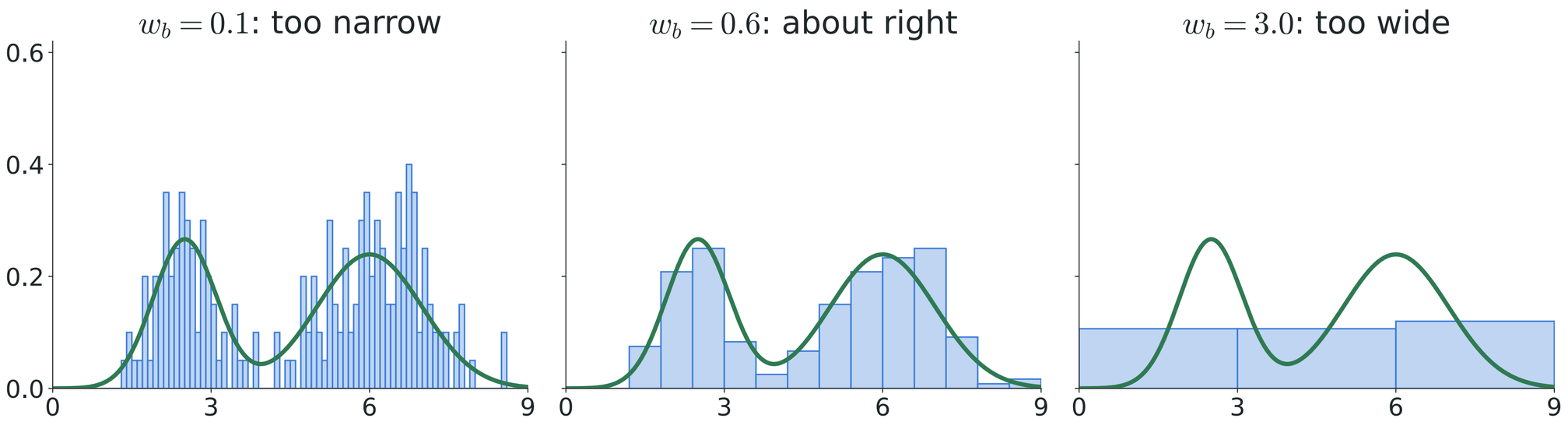
$$\sum_b \hat{p}_b \, w_b = \sum_b \frac{N_b}{N} = 1$$



Here: $\hat{p}_b = \frac{5}{12 \cdot 1.5} = 0.28$ for the highest bin.

! $\hat{p}_b$ is a **height**, not a probability: 6 of 12 samples in a bin of width 0.25 give $\hat{p}_b = 2$

Histogram: pros and cons



Pros

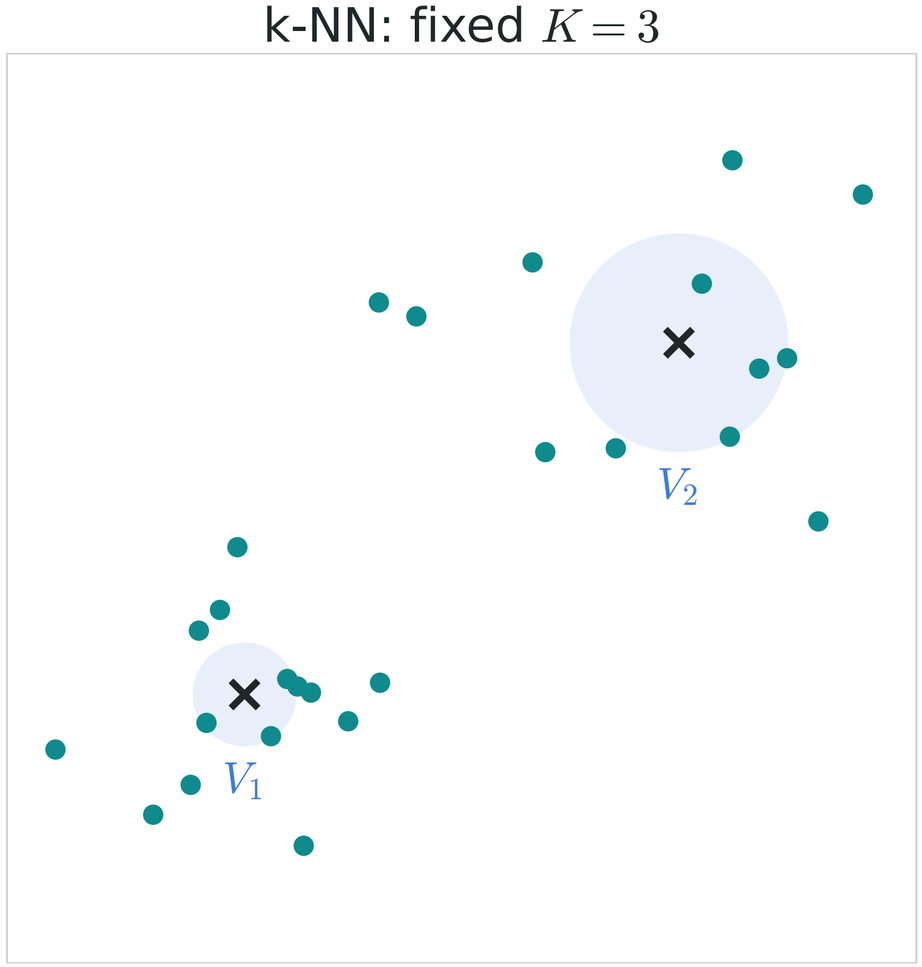
- **Simple**, easy to visualise
- We do not need to **keep** the samples x_i

Cons

- **Step-like**, bins can be **empty**
- How to choose w_b?
- $D > 2$: B^D bins, mostly empty

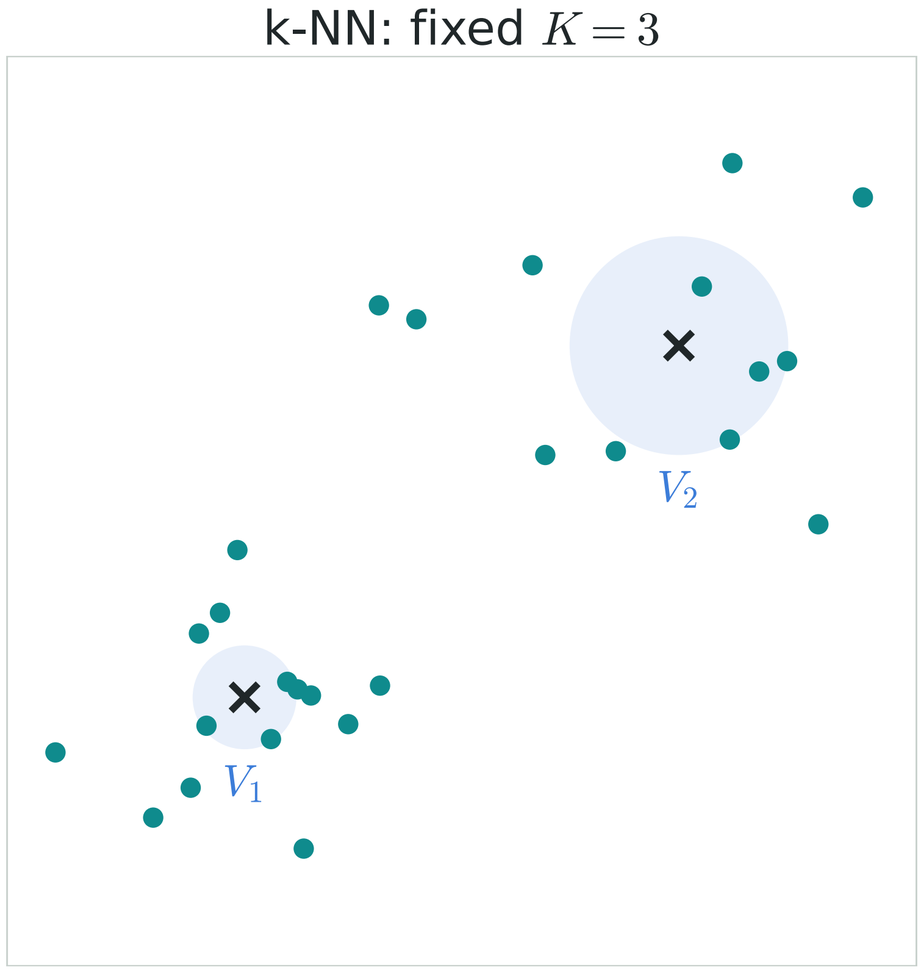
Towards Parzen windows

More dimensions: $\hat{p}(x) = \frac{K}{NV}$



The histogram in D dimensions: K samples in a volume V around x.

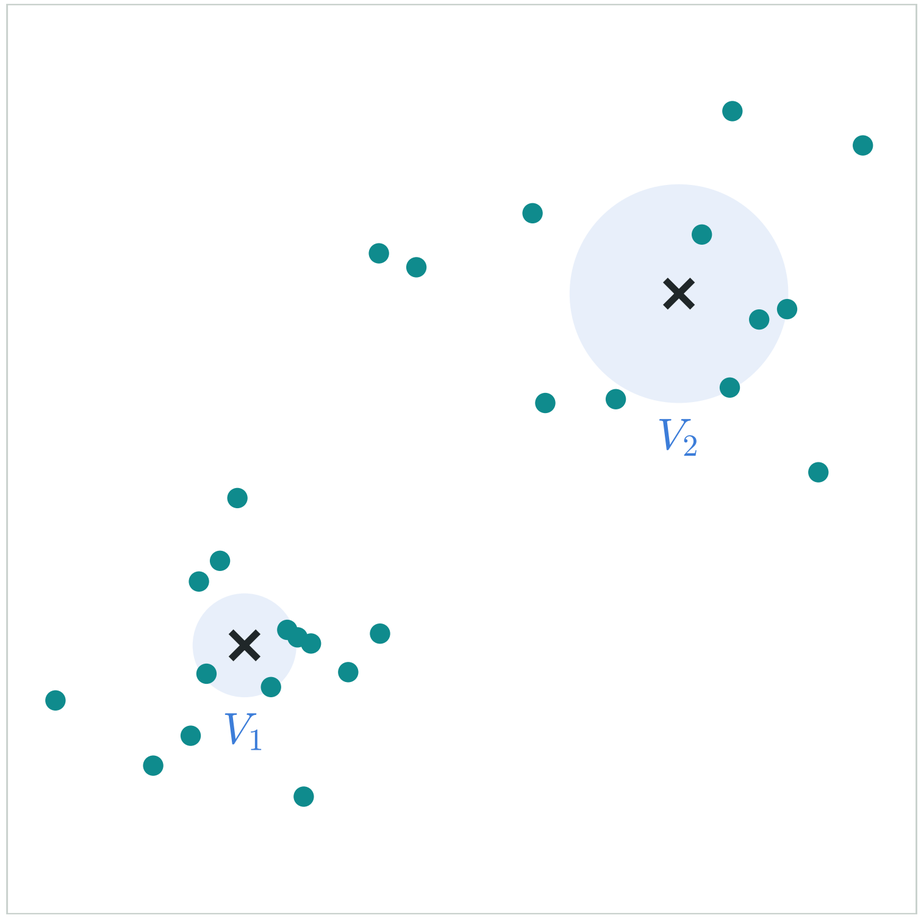
More dimensions: $\hat{p}(x) = \frac{K}{NV}$ $\hat{p}(x) = \frac{K}{NV}$



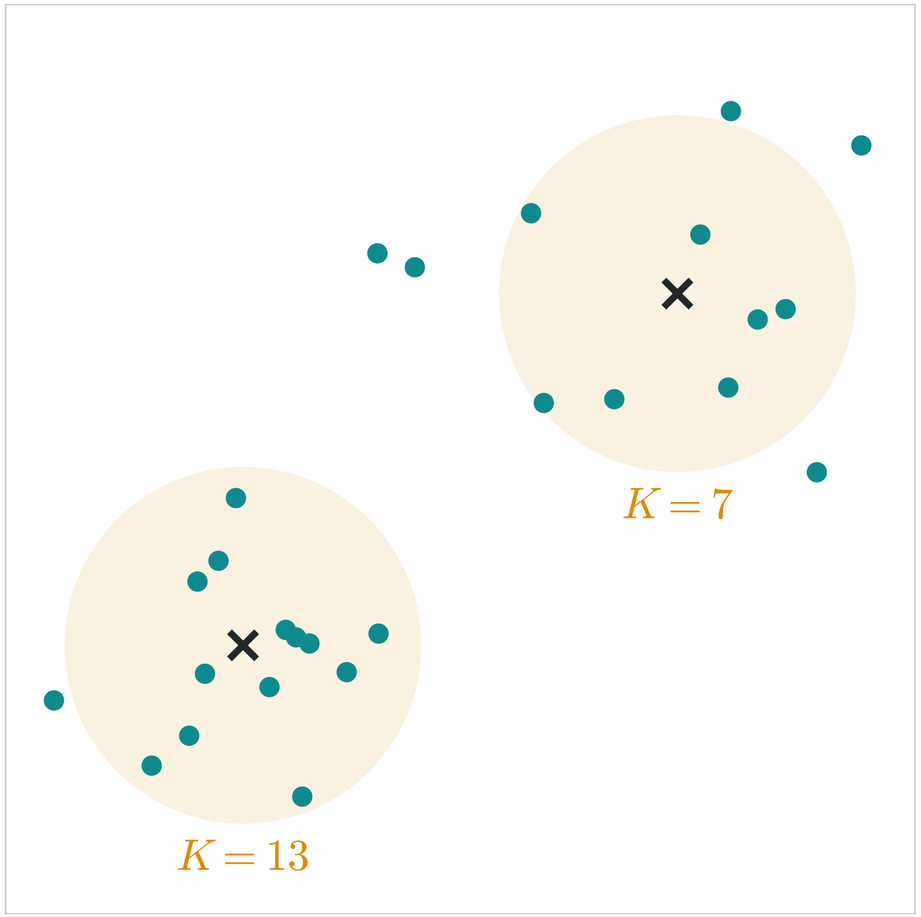
Fix K, find the smallest V containing K samples: k-nearest neighbours.

More dimensions: $\hat{p}(x) = \frac{K}{NV}$ $\hat{p}(x) = \frac{K}{NV}$

k-NN: fixed $K = 3$



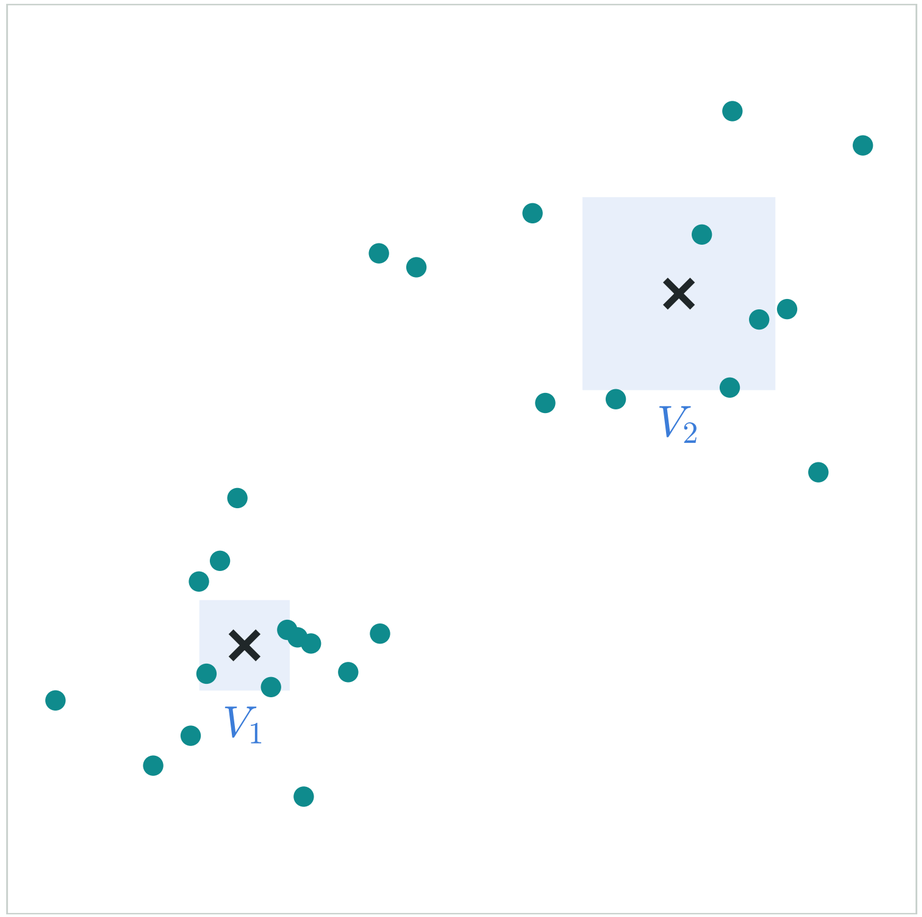
Parzen: fixed V



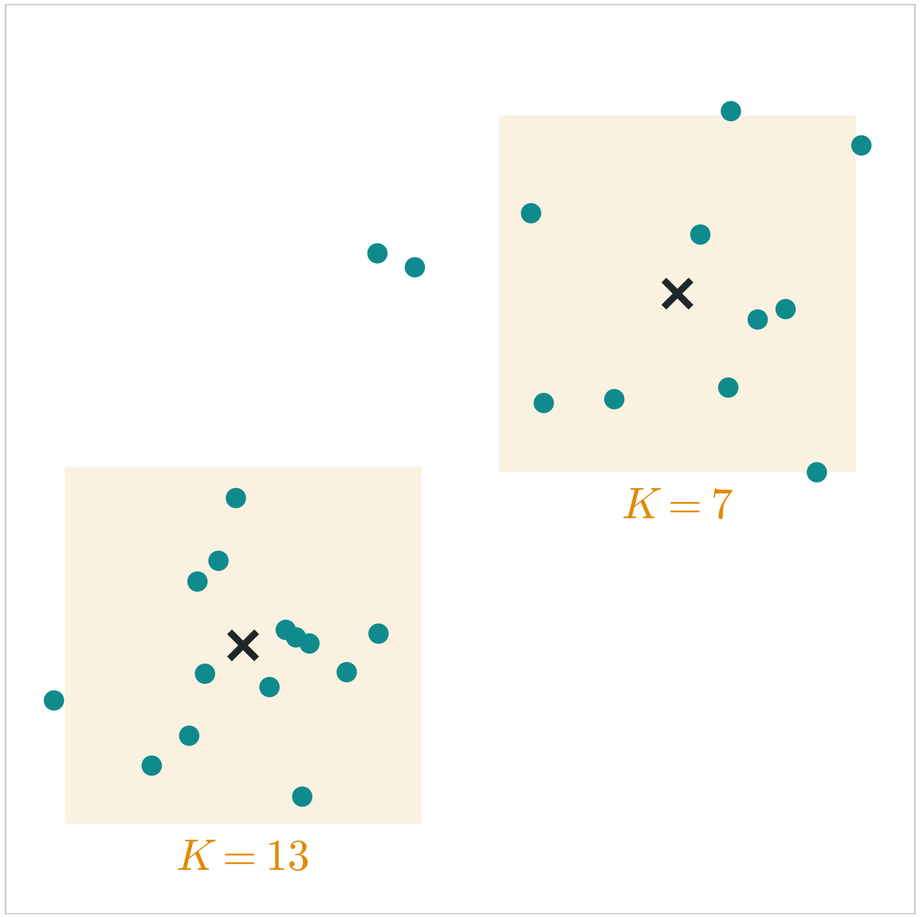
Fix VV, count the samples KK inside: **Parzen windows**.

More dimensions: $\hat{p}(x) = \frac{K}{NV}$ $\hat{p}(x) = \frac{K}{NV}$

k-NN: fixed $K = 3$



Parzen: fixed V



The volume can be a ball or a **hypercube**: the cube is easy to count in.

Counting the samples in a window

Window function: the unit hypercube centred at the origin

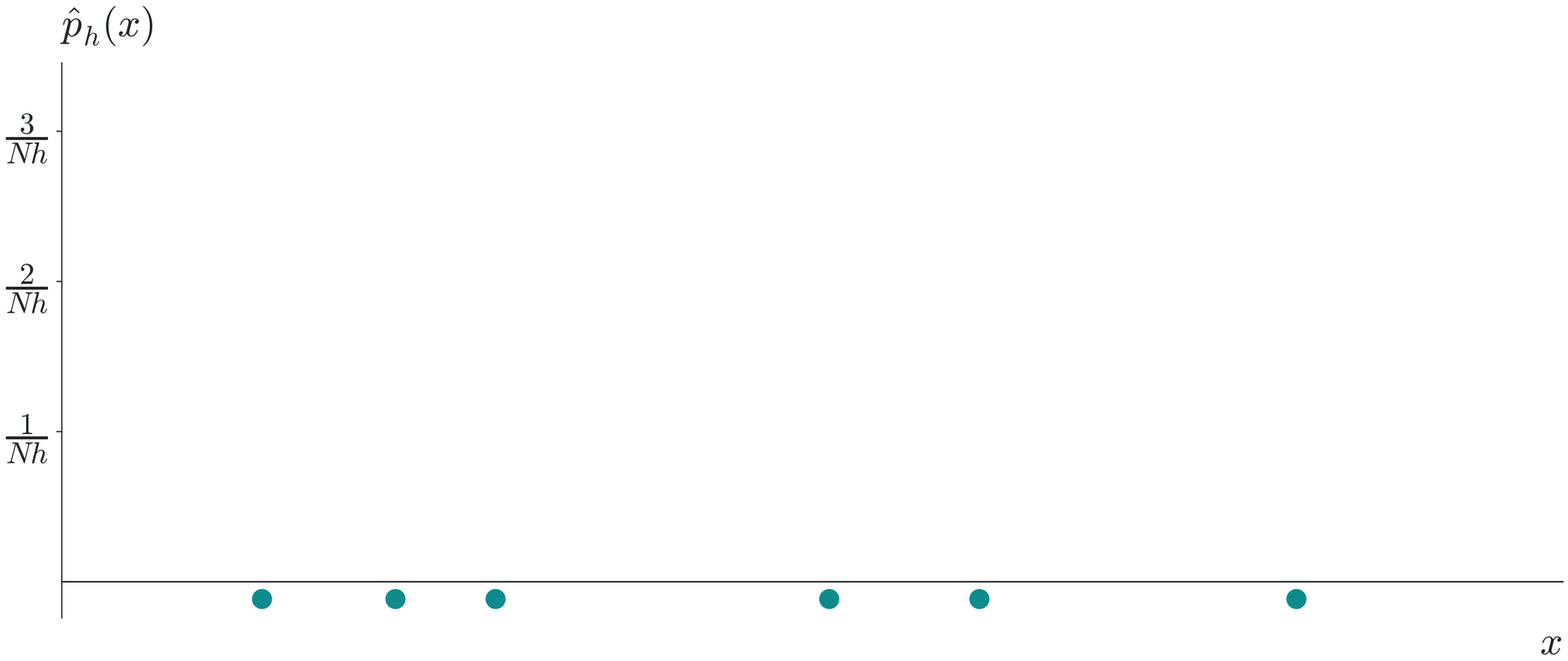
$$\varphi(\mathbf{u}) = \begin{cases} 1 & \text{if } |u^j| \leq \frac{1}{2} \text{ for all } j = 1, \ldots, D \\ 0 & \text{otherwise} \end{cases}$$

$\varphi\left(\frac{x - x_i}{h}\right) = 1$ iff x_i lies in the hypercube of side h around x, so $V = h^D$ and

$$K = \sum_{i=1}^{N} \varphi\left(\frac{x - x_i}{h}\right)$$

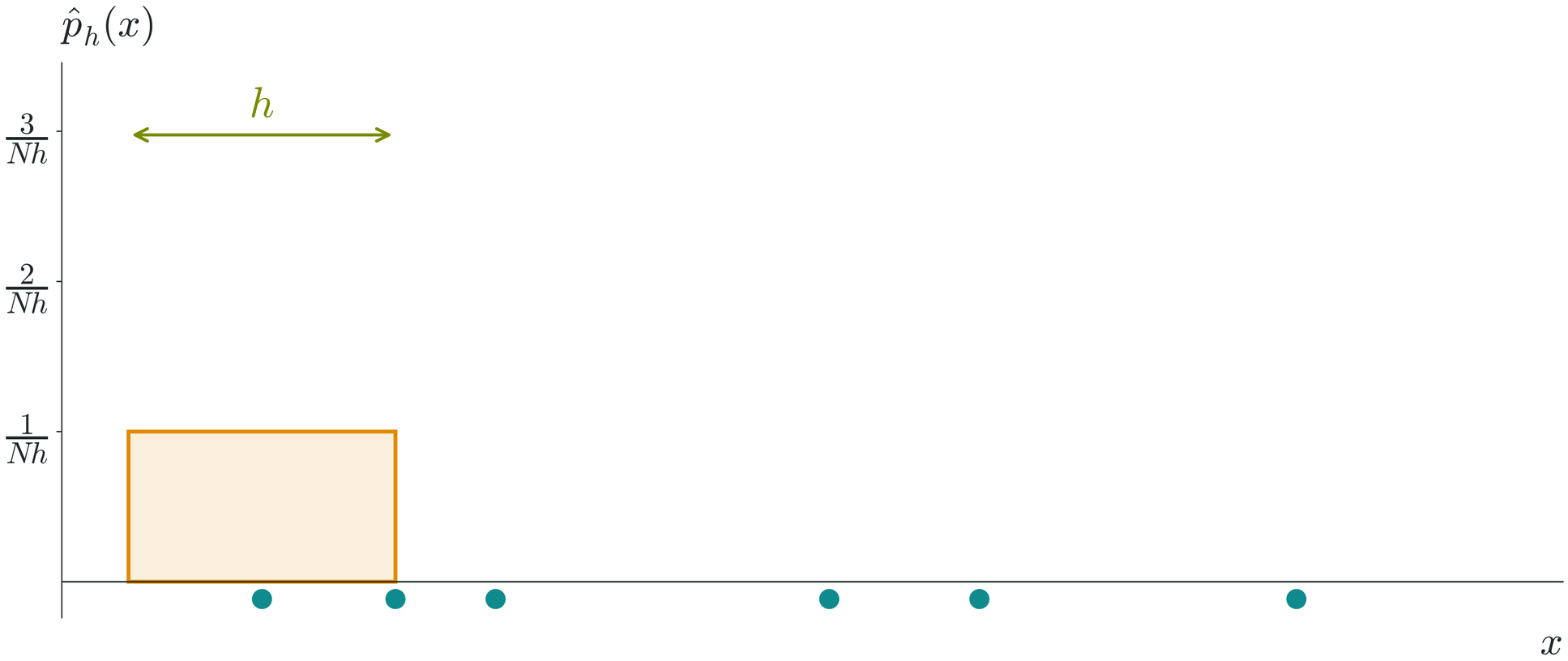
$$\hat{p}_h(x) = \frac{K}{NV} = \frac{1}{N} \sum_{i=1}^{N} \underbrace{\frac{1}{h^D} \varphi\left(\frac{x - x_i}{h}\right)}_{K_h(x, x_i)}$$

A sum of kernels



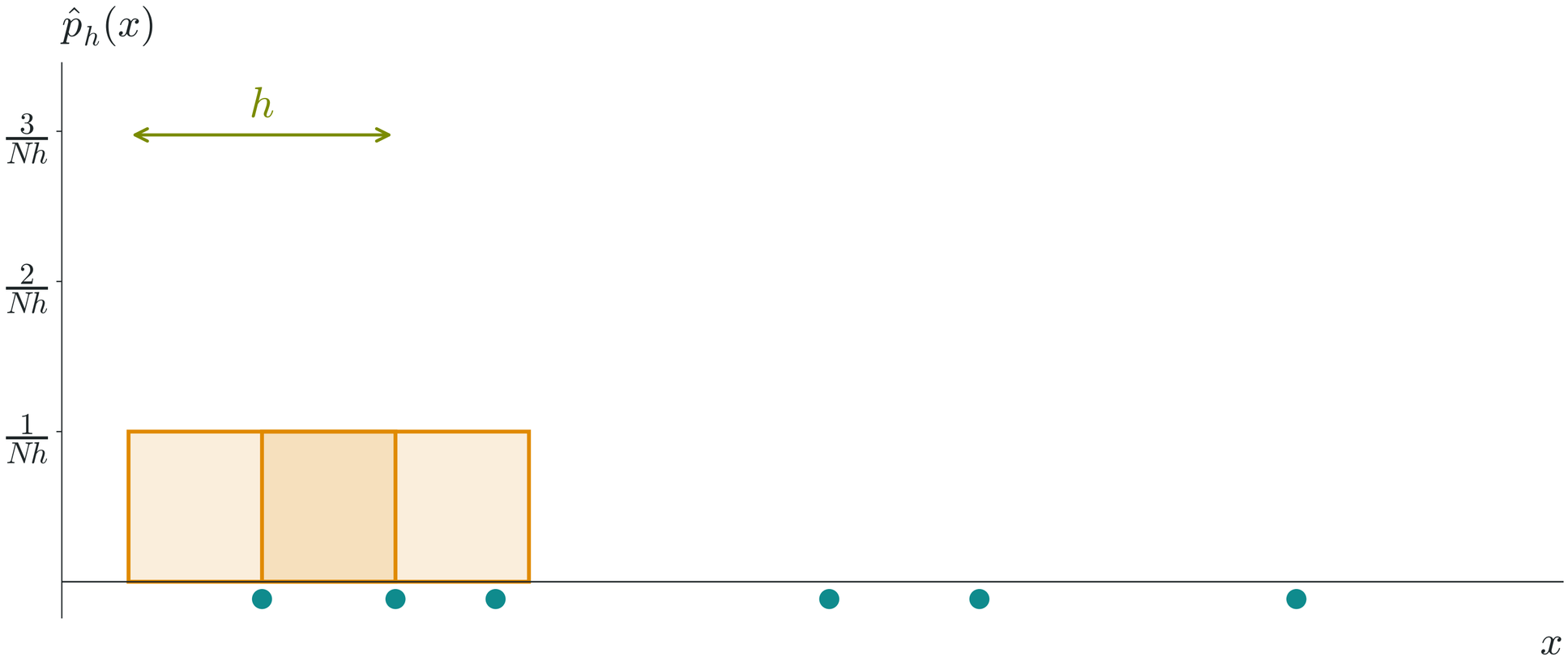
$N = 6$ samples x_i.

A sum of kernels



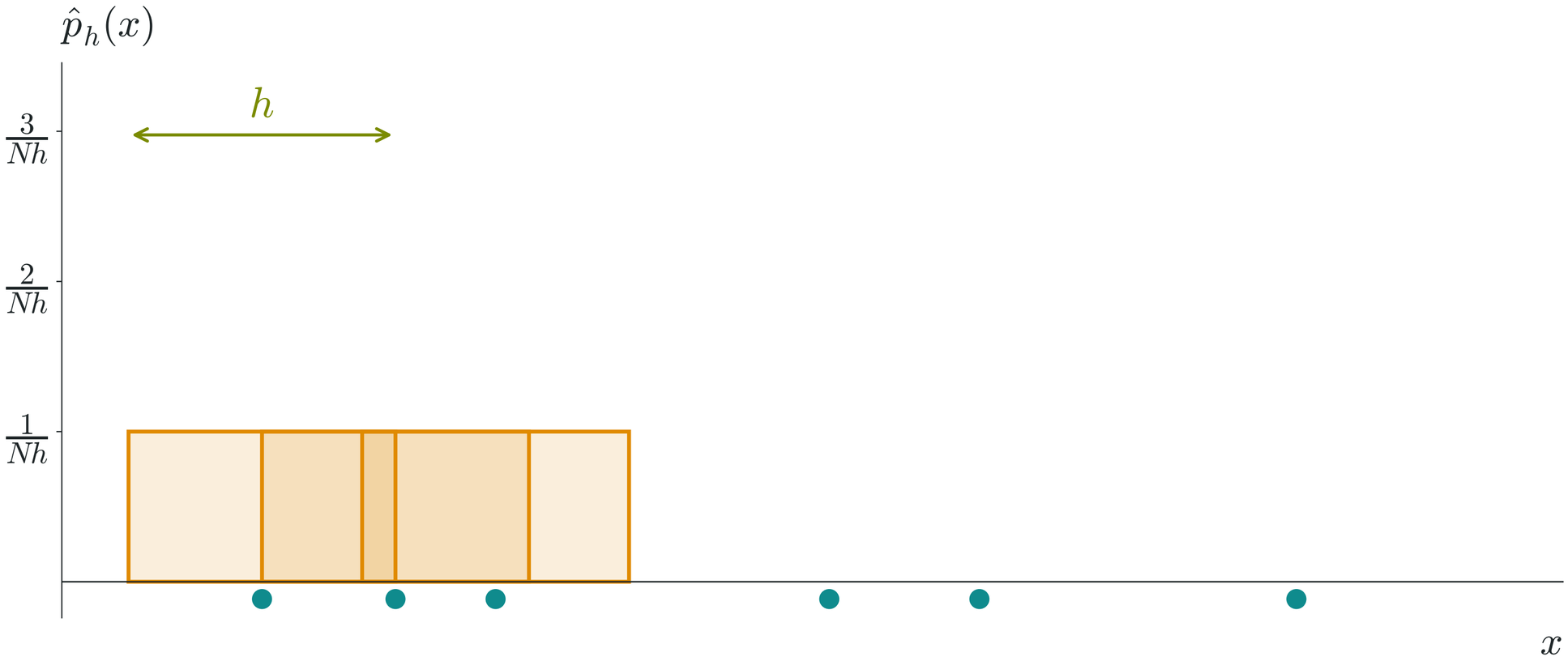
Around every sample a box of width $h$$h$ and height $\frac{1}{Nh}\frac{1}{Nh}$ (area $\frac{1}{N}\frac{1}{N}$).

A sum of kernels



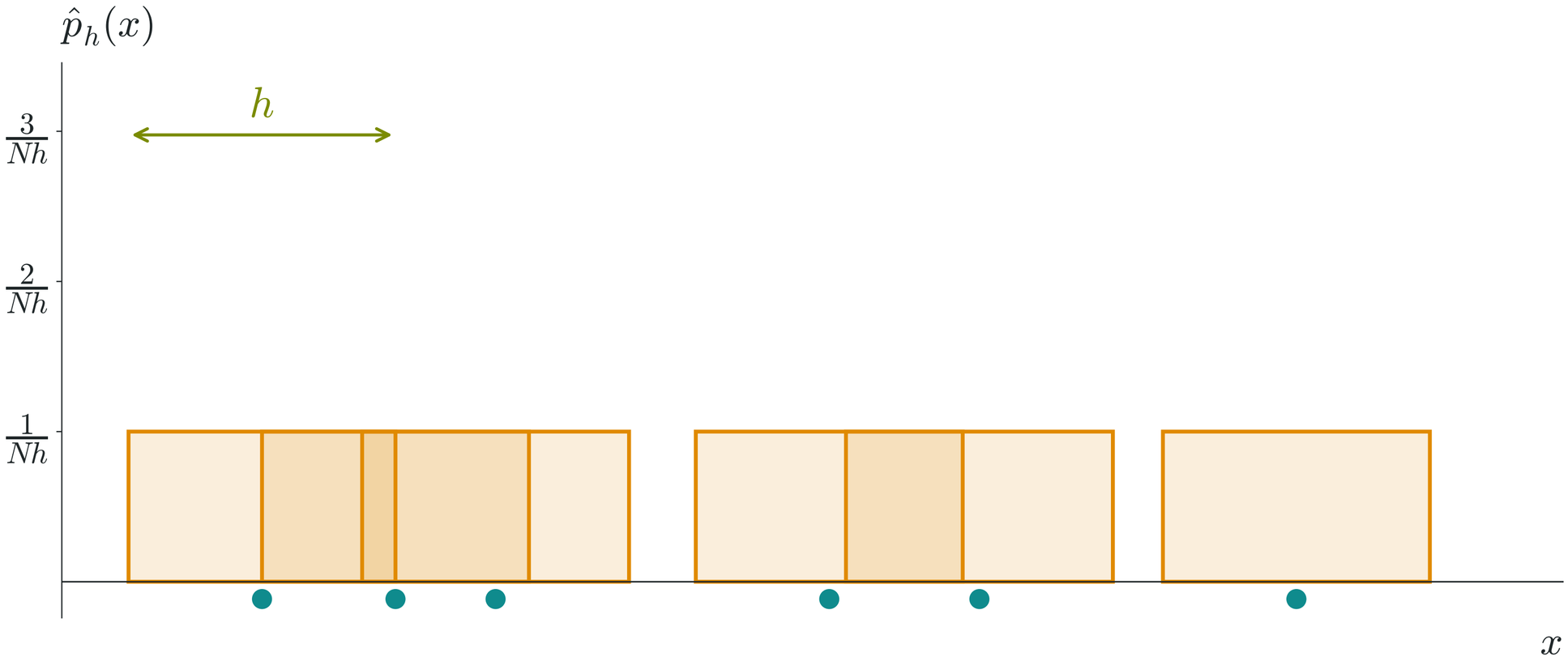
Around every sample a box of width $h$$h$ and height $\frac{1}{Nh}\frac{1}{Nh}$ (area $\frac{1}{N}\frac{1}{N}$).

A sum of kernels



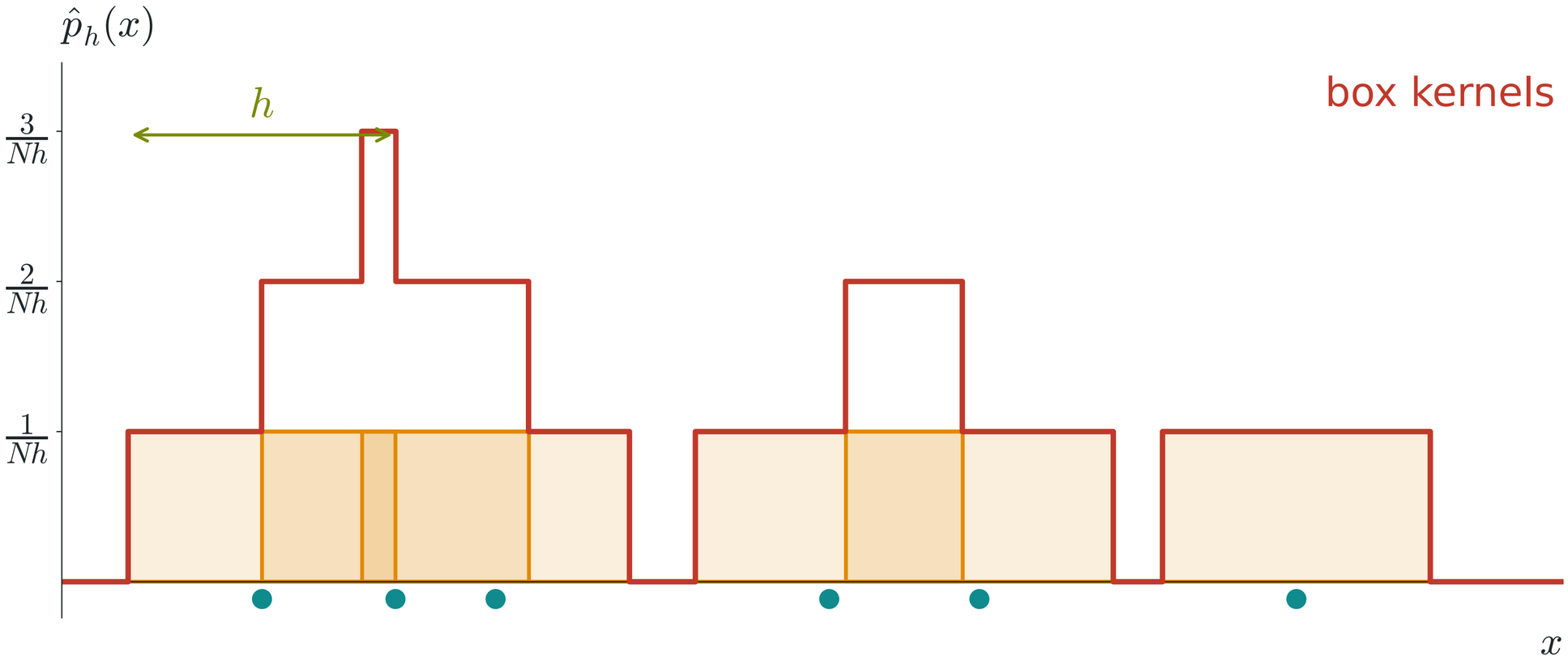
Around every sample a box of width $h$$h$ and height $\frac{1}{Nh}\frac{1}{Nh}$ (area $\frac{1}{N}\frac{1}{N}$).

A sum of kernels



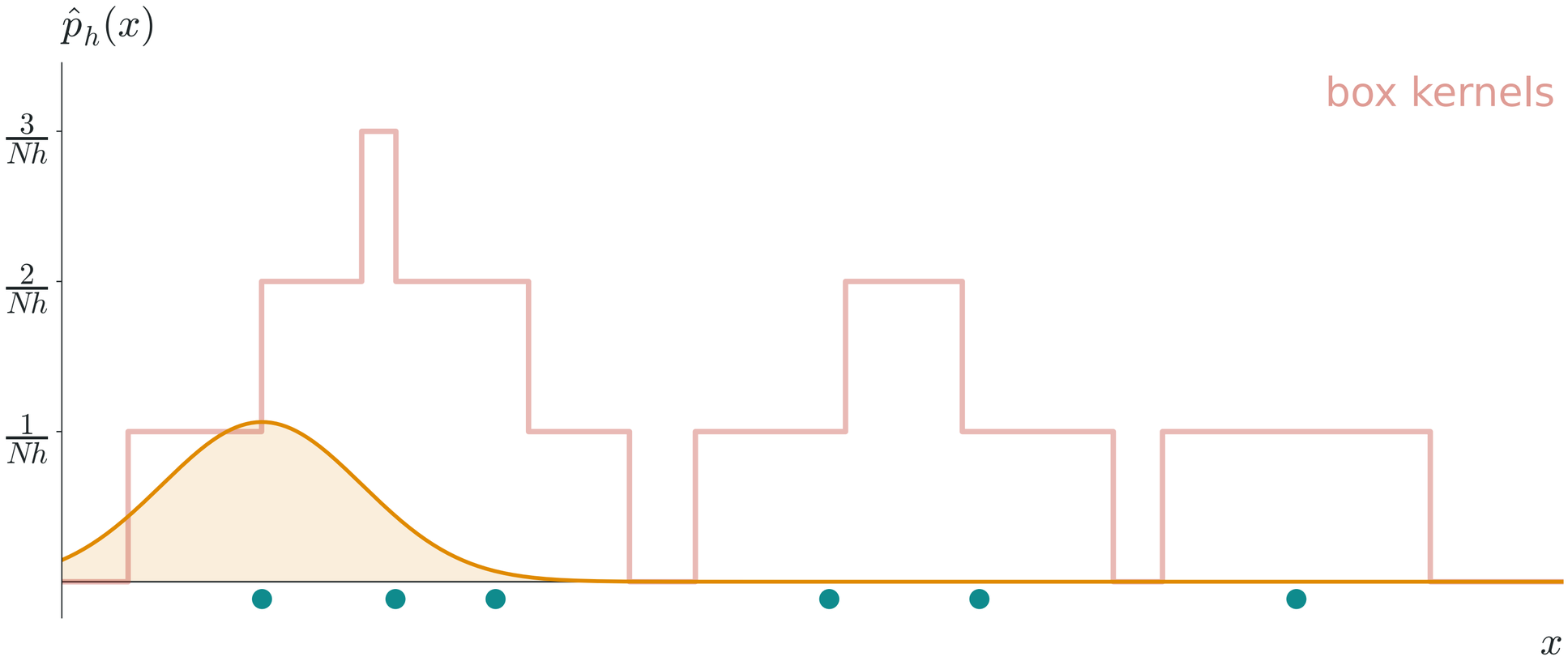
Around every sample a box of width h and height $\frac{1}{Nh}\frac{1}{Nh}$ (area $\frac{1}{N}\frac{1}{N}$).

A sum of kernels



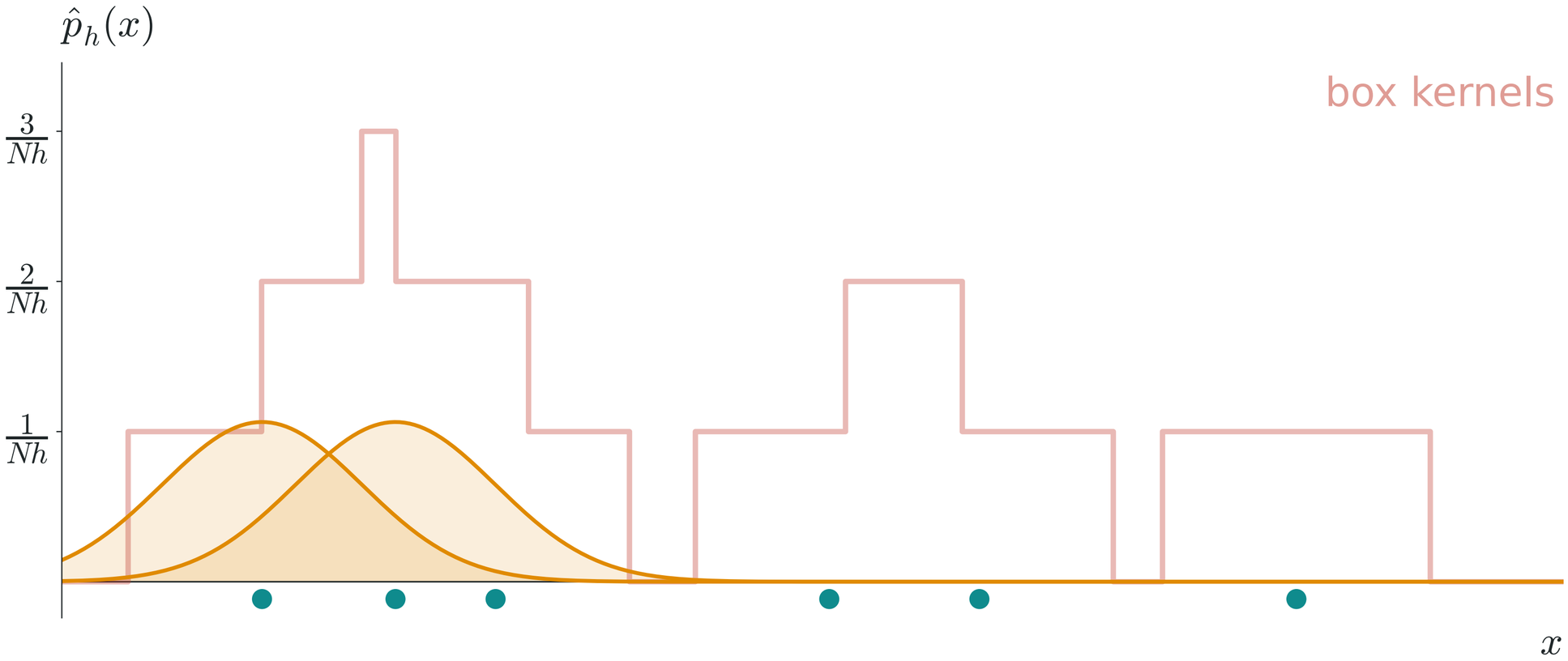
$\hat{p}_h(x)\hat{p}_h(x)$ is their **sum**: still step-like, but no fixed bins.

A sum of kernels



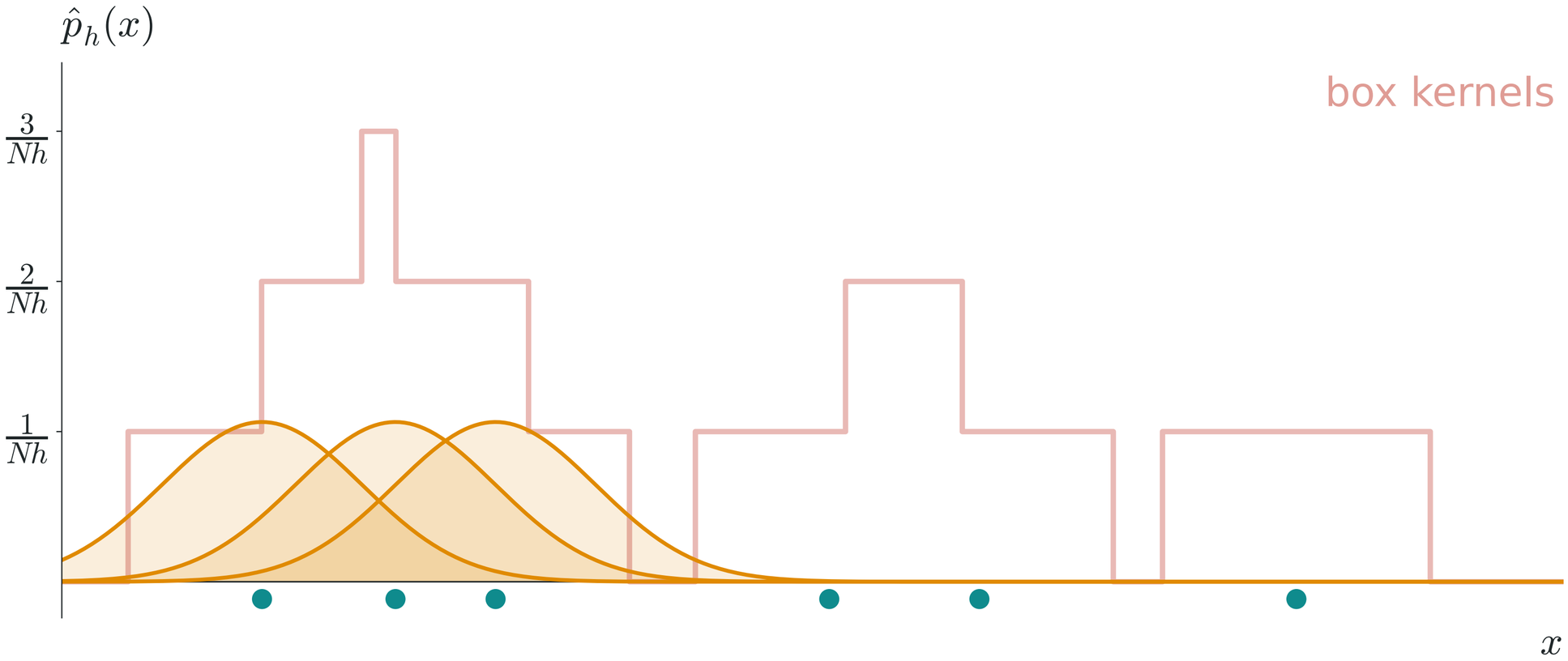
A **Gaussian kernel** around every sample, again with area $\frac{1}{N}\frac{1}{N}$.

A sum of kernels



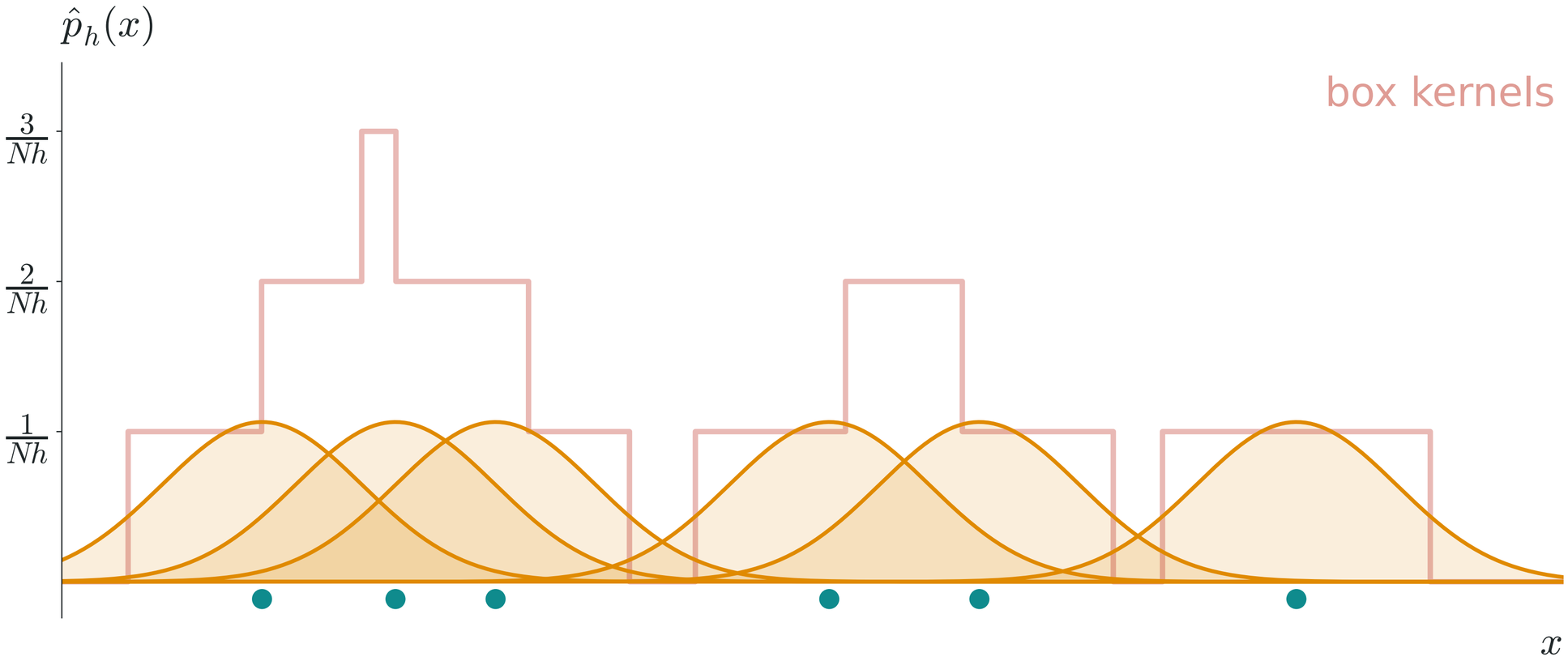
A **Gaussian kernel** around every sample, again with area $\frac{1}{N}\frac{1}{N}$.

A sum of kernels



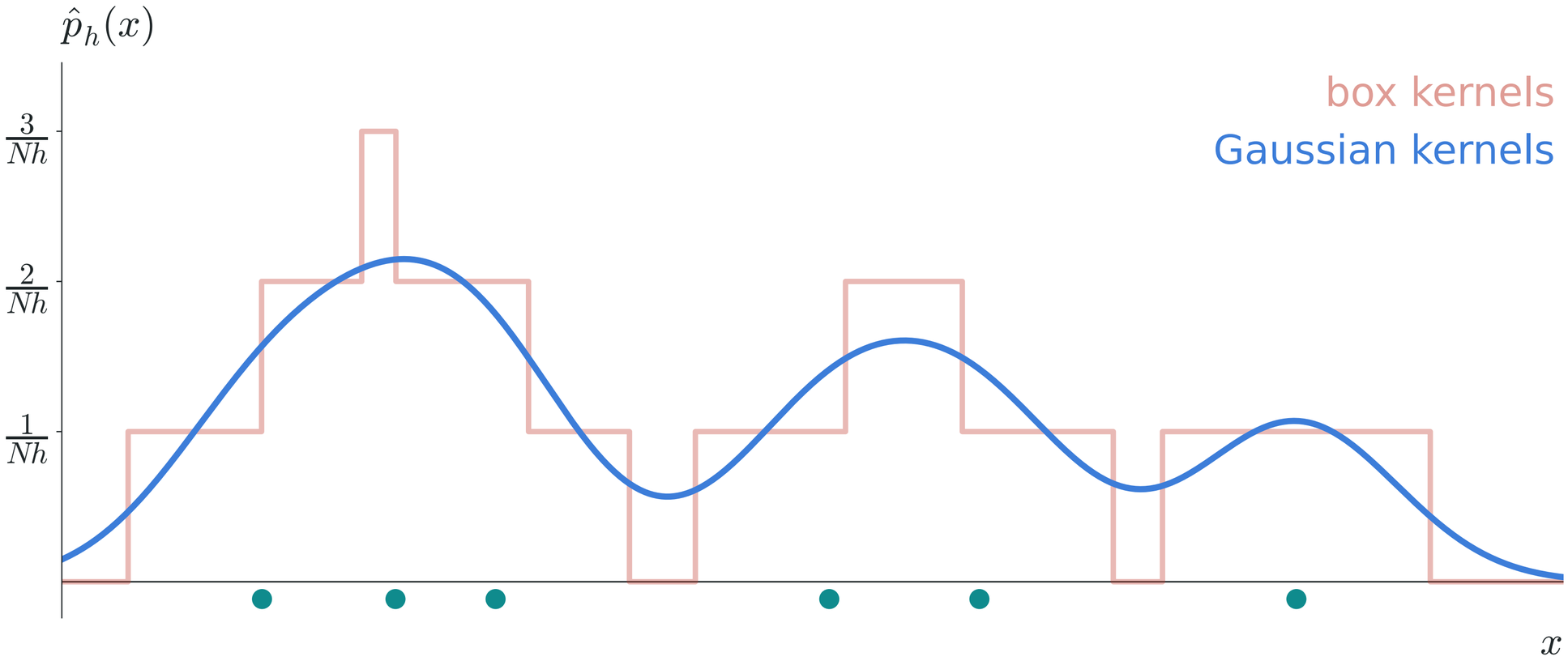
A **Gaussian kernel** around every sample, again with area $\frac{1}{N}\frac{1}{N}$.

A sum of kernels



A **Gaussian kernel** around every sample, again with area $\frac{1}{N}\frac{1}{N}$.

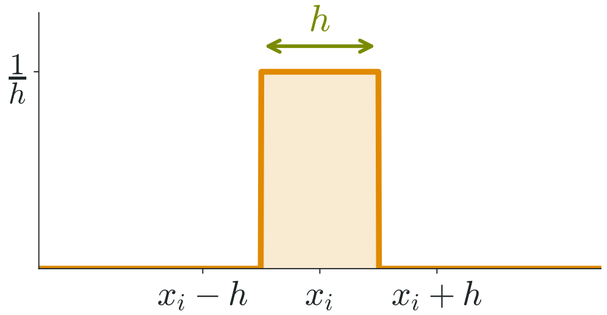
A sum of kernels



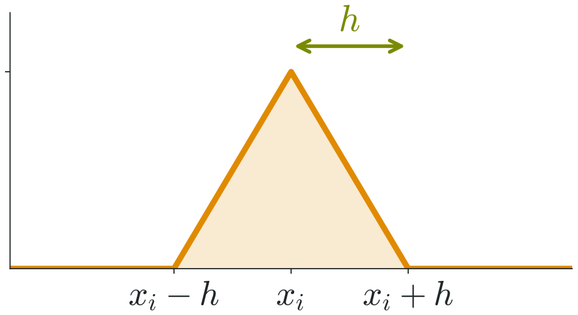
Their sum is a **smooth** estimate.

Kernels $K_{\text{type}}(x, x_i)$

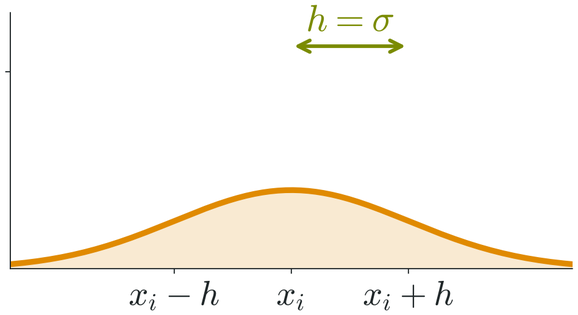
box



triangle



Gaussian



$$K_{\text{box}}(x, x_i) = \begin{cases} \frac{1}{h} & \text{if } |x - x_i| \le \frac{h}{2} \\ 0 & \text{otherwise} \end{cases} \qquad \int K_{\text{box}}(x, x_i)\, dx = h \cdot \frac{1}{h} = 1$$

$$K_{\text{tri}}(x, x_i) = \begin{cases} \frac{1}{h}\left(1 - \frac{|x - x_i|}{h}\right) & \text{if } |x - x_i| \le h \\ 0 & \text{otherwise} \end{cases} \qquad \int K_{\text{tri}}(x, x_i)\, dx = \frac{1}{2} \cdot 2h \cdot \frac{1}{h} = 1$$

$$K_{\text{Gauss}}(x, x_i) = \frac{1}{\sqrt{2\pi}\, h}\, e^{-\frac{(x - x_i)^2}{2h^2}} \qquad \int K_{\text{Gauss}}(x, x_i)\, dx = 1$$

What makes a kernel?

Which properties must $K_{\text{type}}(x, x_i)$ have, so that the estimate $\hat{p}_h(x)$ is a density?

For all x and x_i:

$$K_{\text{type}}(x, x_i) \geq 0, \qquad \int K_{\text{type}}(x, x_i)\, dx = 1$$

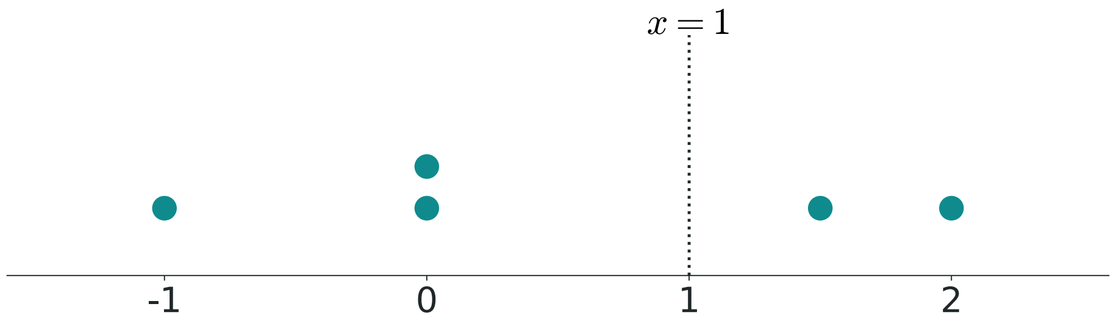
$$\int \hat{p}_h(x)\, dx = \int \frac{1}{N} \sum_{i=1}^{N} K_{\text{type}}(x, x_i)\, dx = \frac{1}{N} \sum_{i=1}^{N} \int K_{\text{type}}(x, x_i)\, dx = \frac{1}{N} \cdot N = 1$$

Usually also symmetric:

$$K_{\text{type}}(x, x_i) = K_{\text{type}}(x_i, x)$$

Assignment: $K_{\text{Gauss}}(x, x_i)$ = `norm.pdf(x, loc=x_i, scale=h)`

Example: estimate by hand



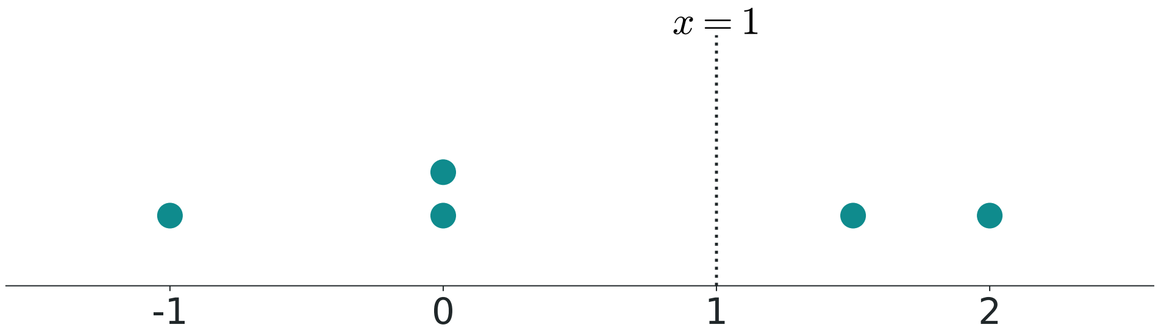
Samples: $-1,\ 0,\ 0,\ 1.5,\ 2 \quad (N = 5)$

1. Box kernel, $h = 1$: what is $\hat{p}_h(1)$?

2. The same for $h = 2$.

3. Which h gives a larger estimate far from all samples, e.g. at $x = 5$?

Derivation: estimate by hand



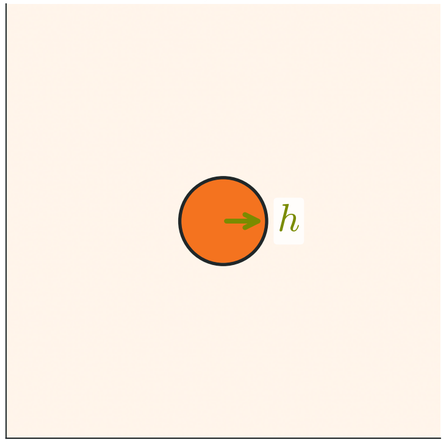
Derivation: estimate by hand

Derivation: estimate by hand

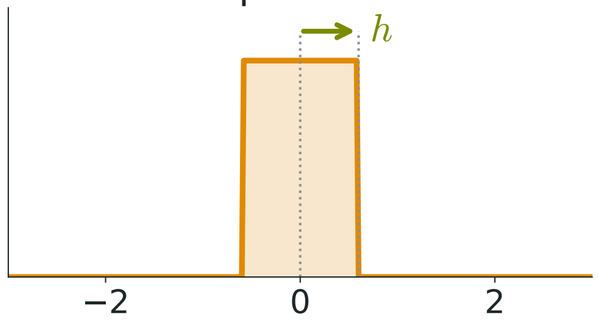
Derivation: estimate by hand

Parzen windows in 2D

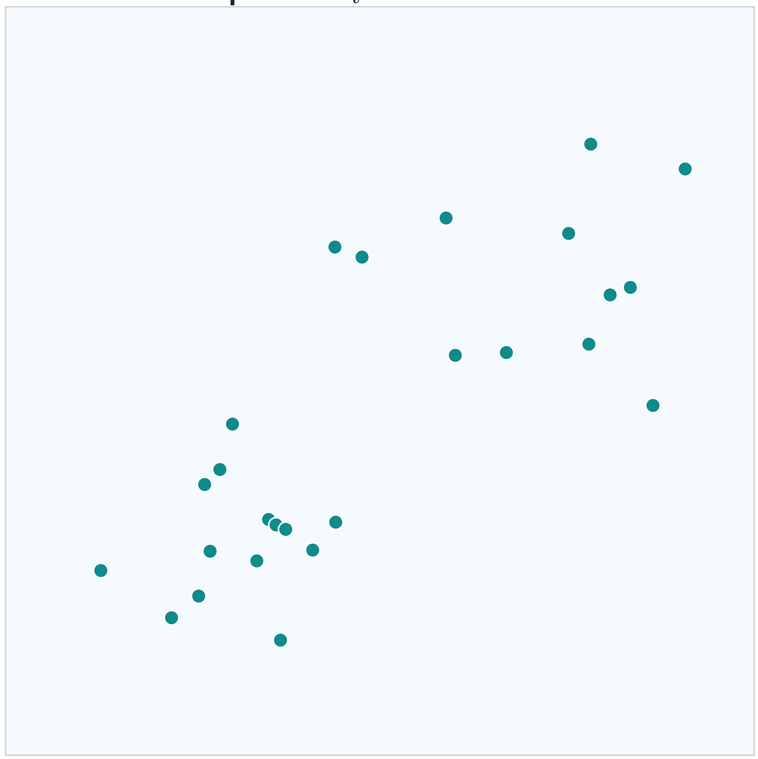
ball kernel, from above



profile



samples x_i and kernels

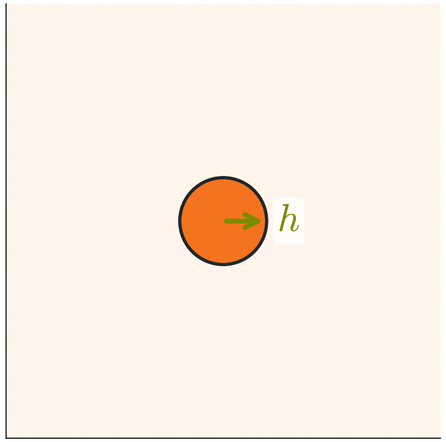


estimate $\hat{p}_h(x)$

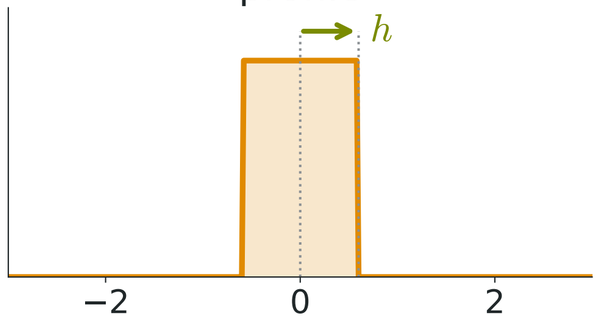
Ball kernel, small h (radius). Put it on every sample and average.

Parzen windows in 2D

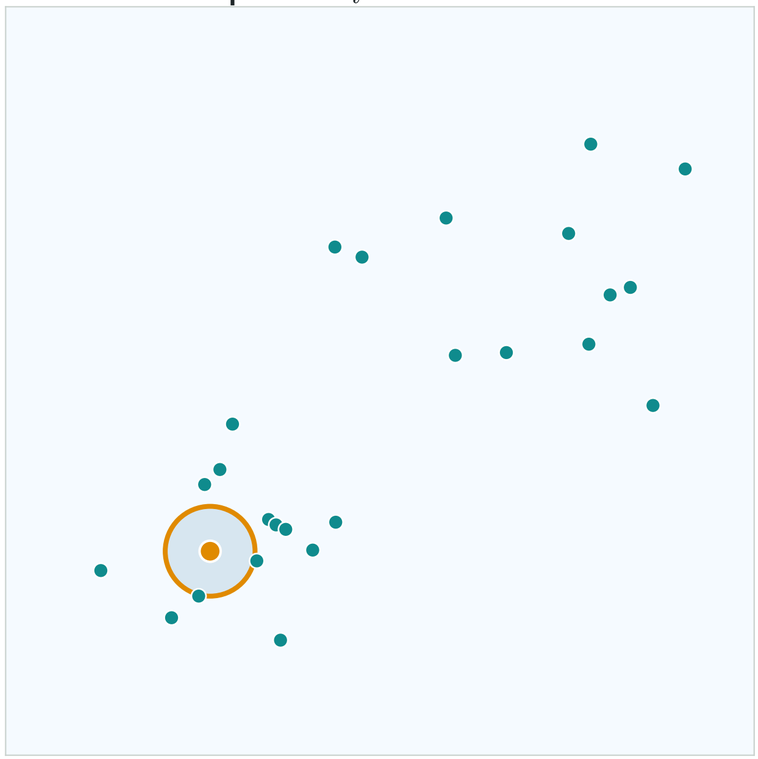
ball kernel, from above



profile



samples x_i and kernels

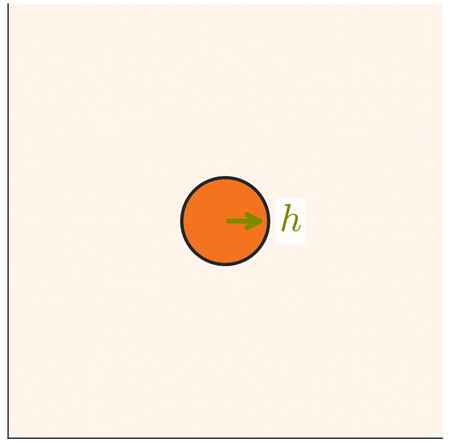


estimate $\hat{p}_h(x)$

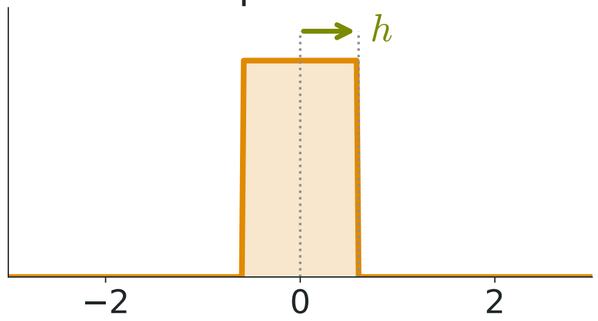
Ball kernel, small h (radius). Put it on every sample and average.

Parzen windows in 2D

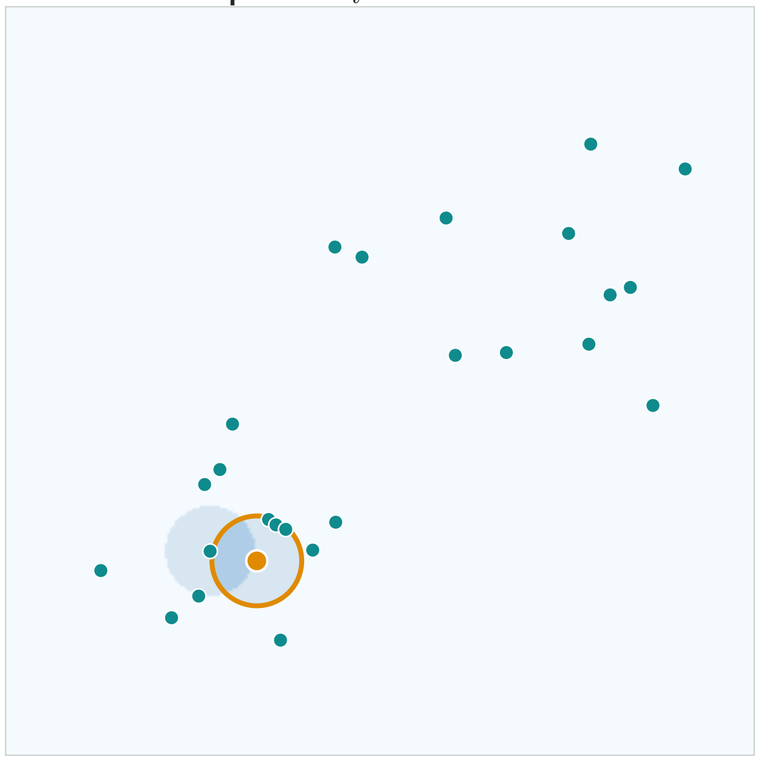
ball kernel, from above



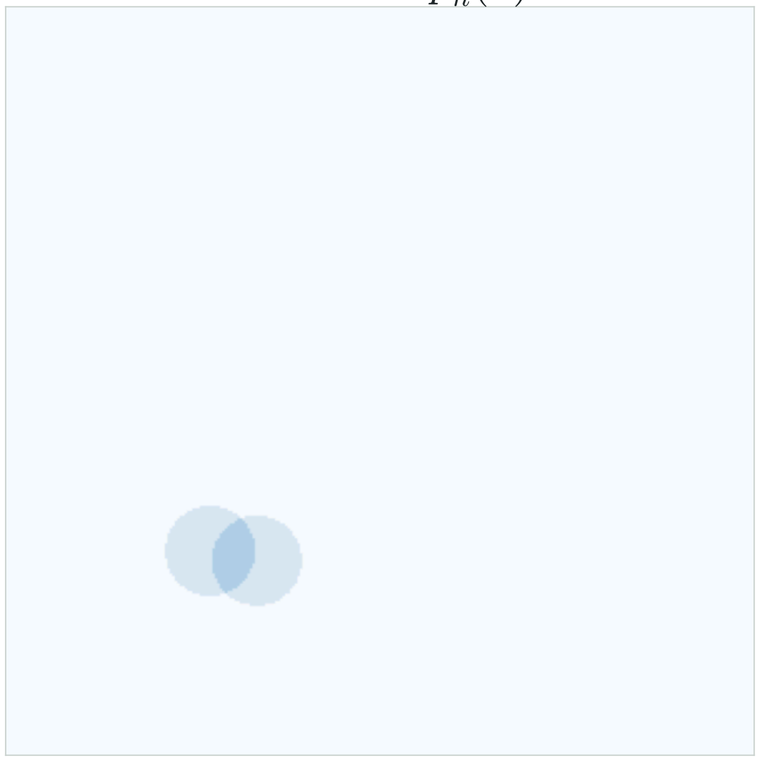
profile



samples x_i and kernels



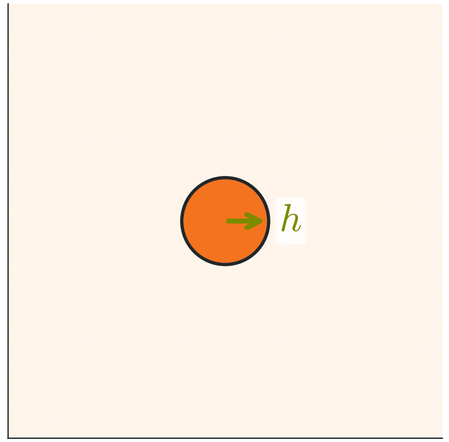
estimate $\hat{p}_h(x)$



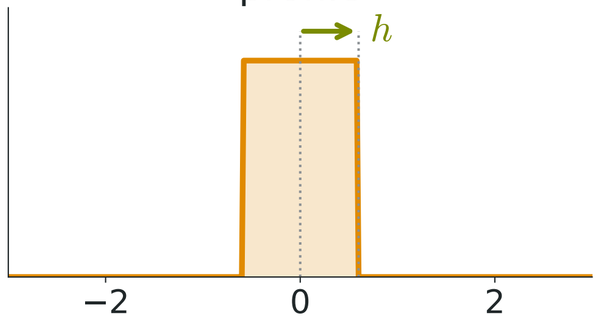
Ball kernel, small h (radius). Put it on every sample and average.

Parzen windows in 2D

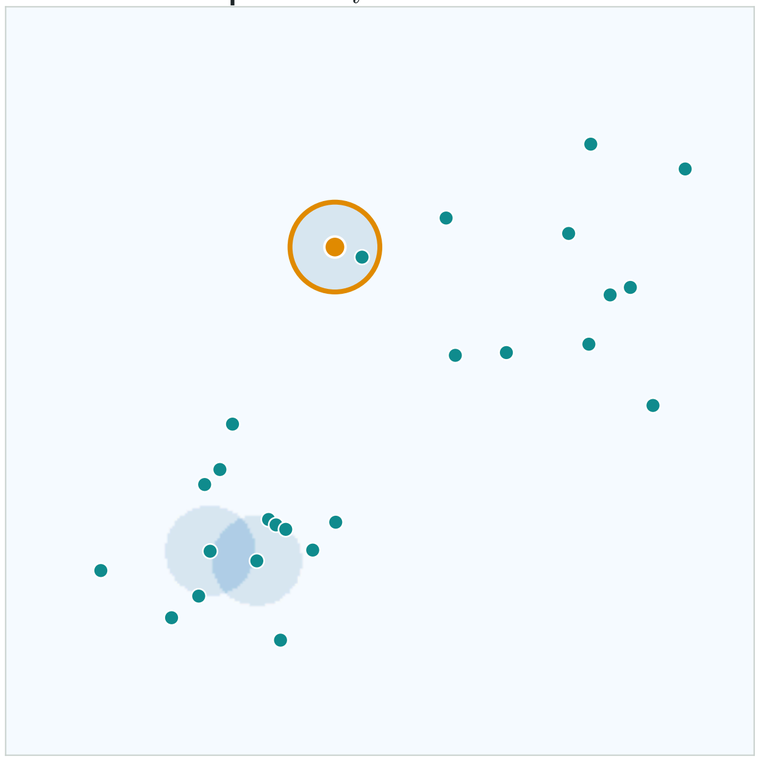
ball kernel, from above



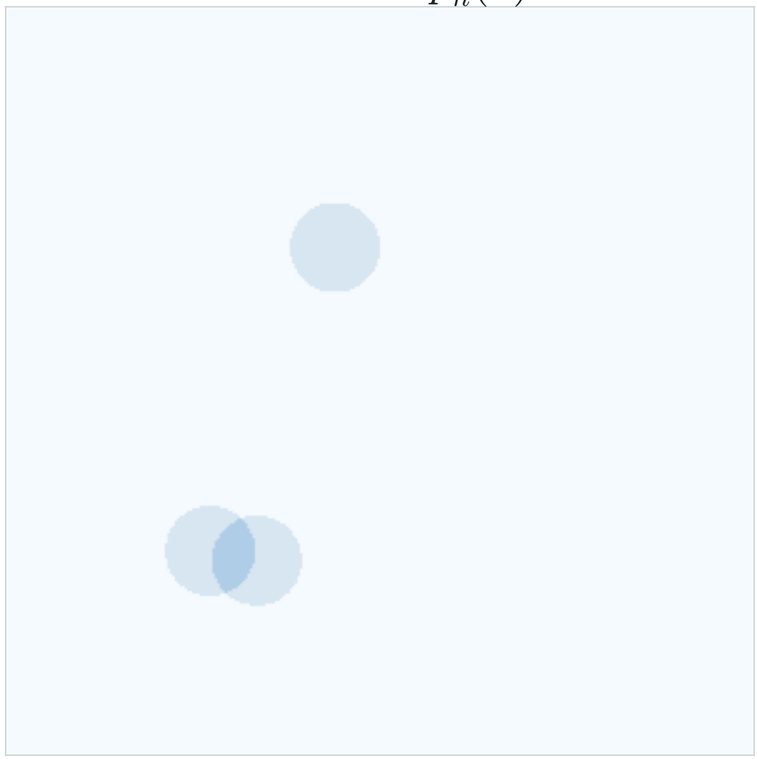
profile



samples x_i and kernels



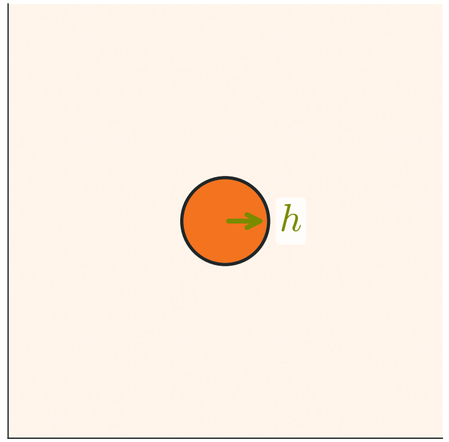
estimate $\hat{p}_h(x)$



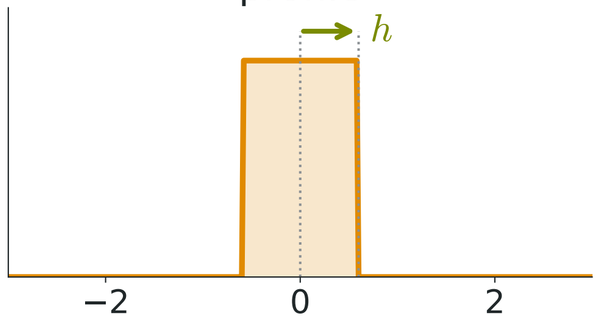
Ball kernel, small $h$$h$ (radius). Put it on every sample and average.

Parzen windows in 2D

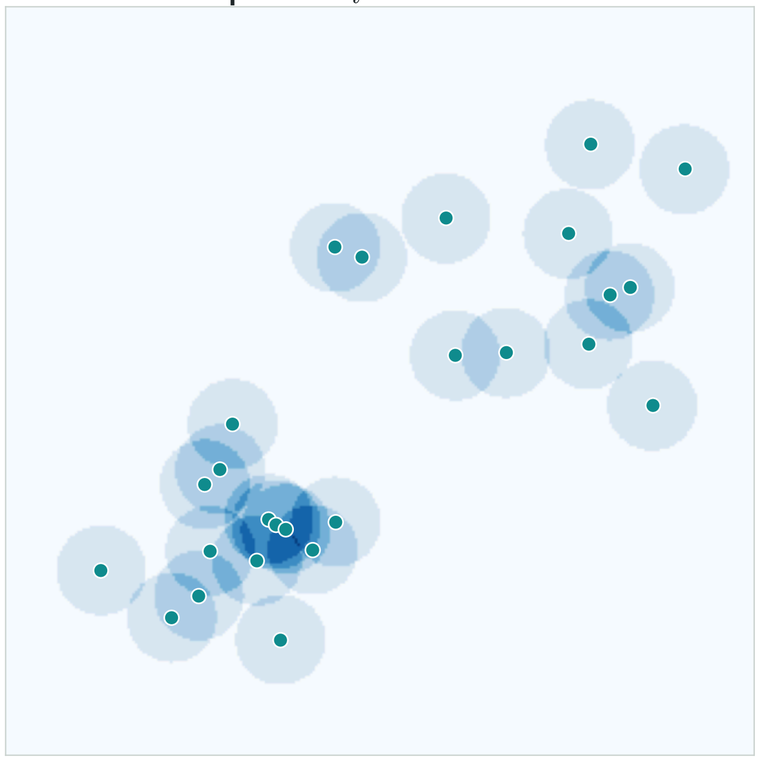
ball kernel, from above



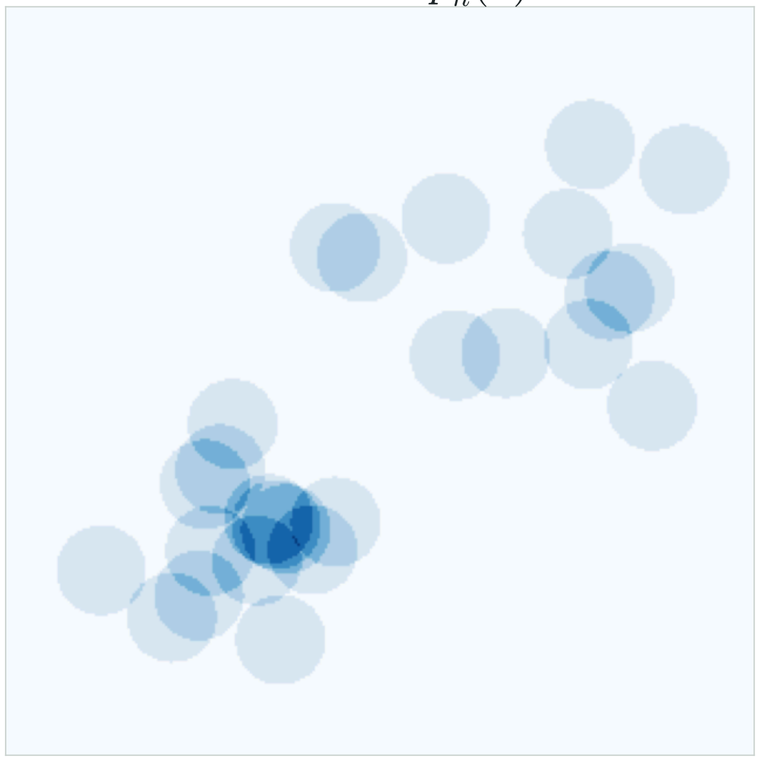
profile



samples x_i and kernels



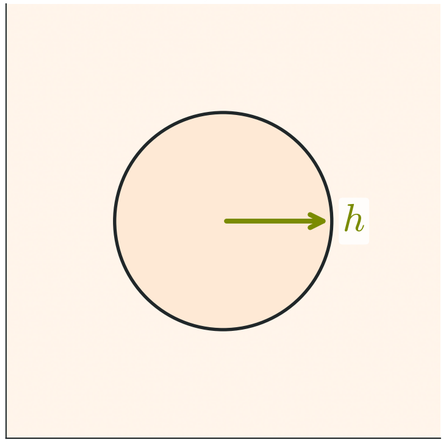
estimate $\hat{p}_h(x)$



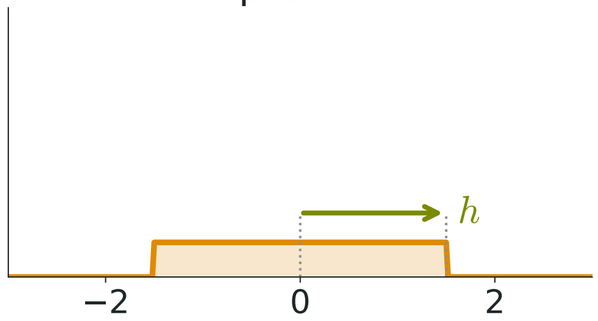
Small h: a separate disc around every sample, gaps in between.

Parzen windows in 2D

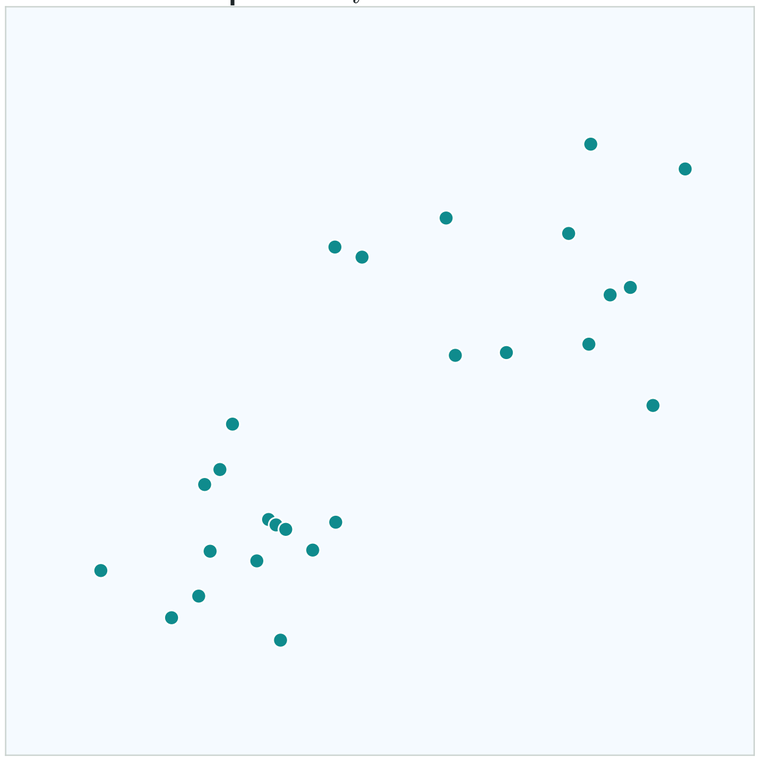
ball kernel, from above



profile



samples x_i and kernels

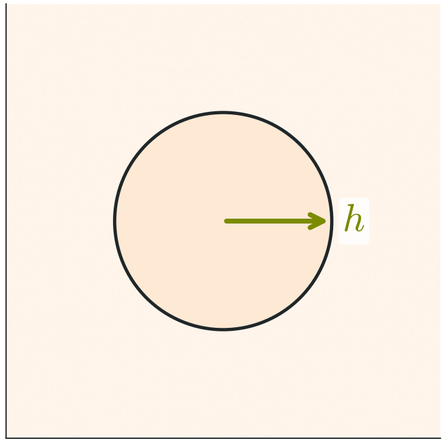


estimate $\hat{p}_h(x)$

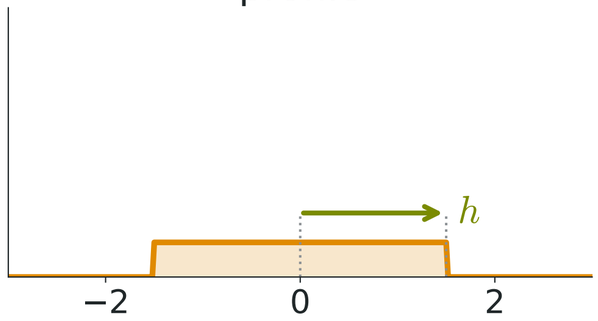
Ball kernel, large hh.

Parzen windows in 2D

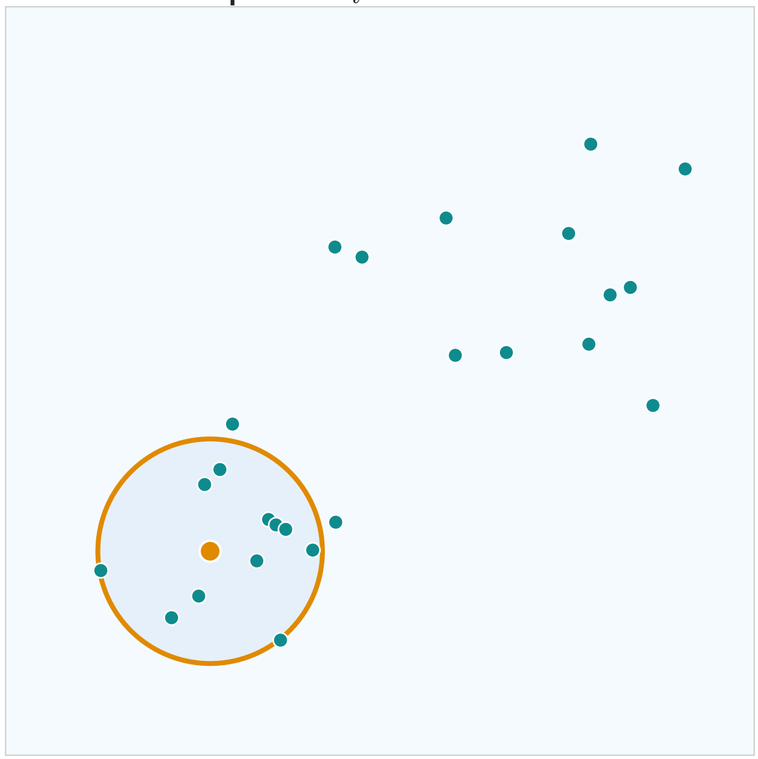
ball kernel, from above



profile



samples x_i and kernels

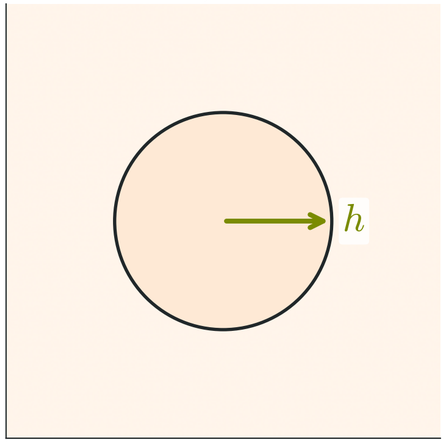


estimate $\hat{p}_h(x)$

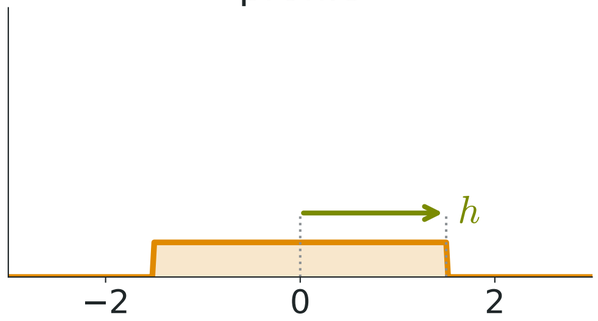
Ball kernel, large hh.

Parzen windows in 2D

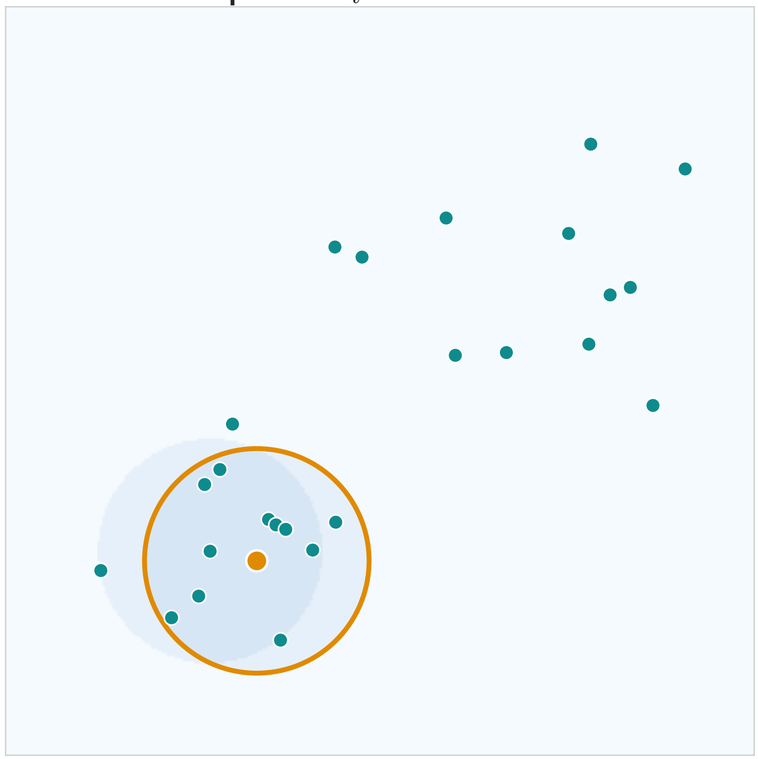
ball kernel, from above



profile



samples x_i and kernels

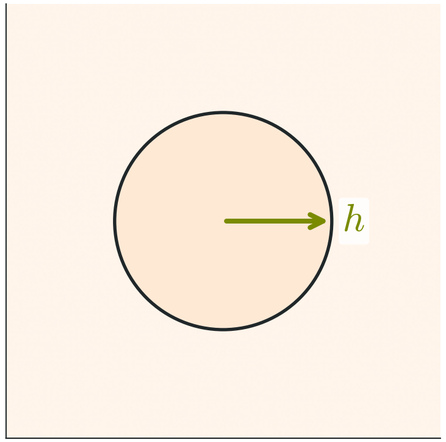


estimate $\hat{p}_h(x)$

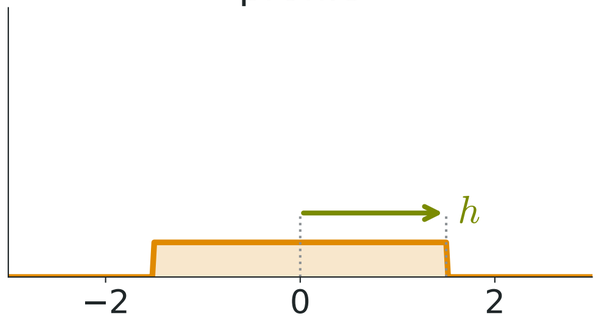
Ball kernel, large hh.

Parzen windows in 2D

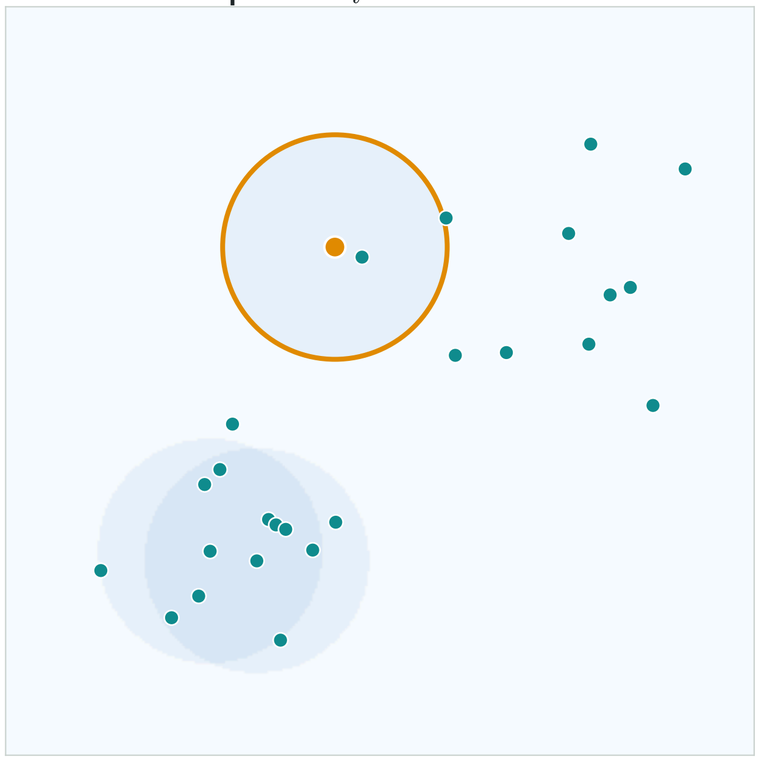
ball kernel, from above



profile



samples x_i and kernels

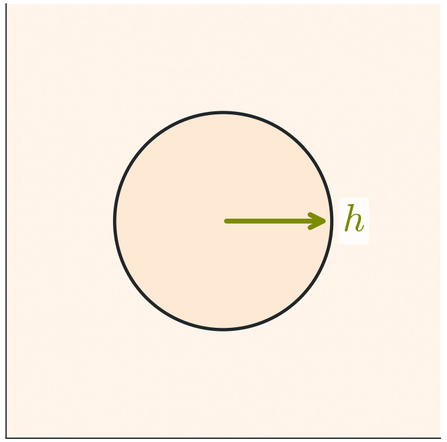


estimate $\hat{p}_h(x)$

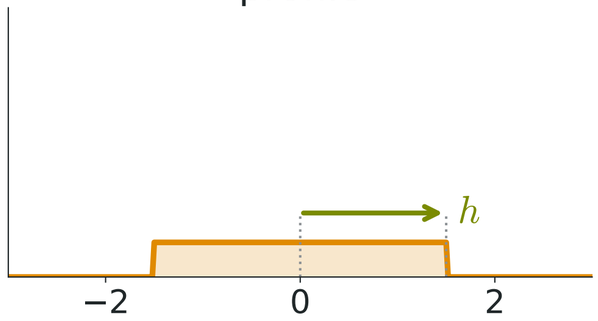
Ball kernel, large $h$$h$.

Parzen windows in 2D

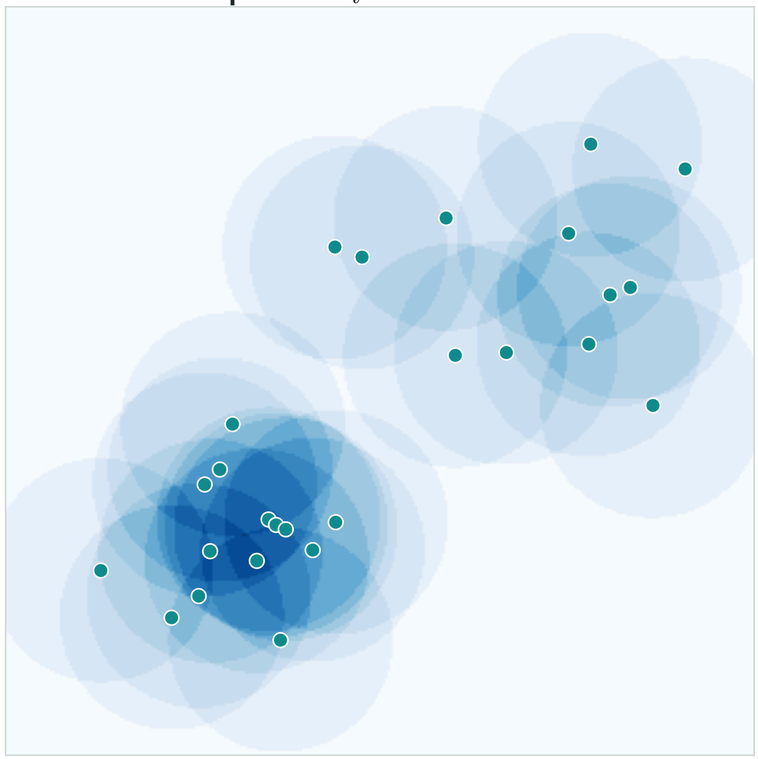
ball kernel, from above



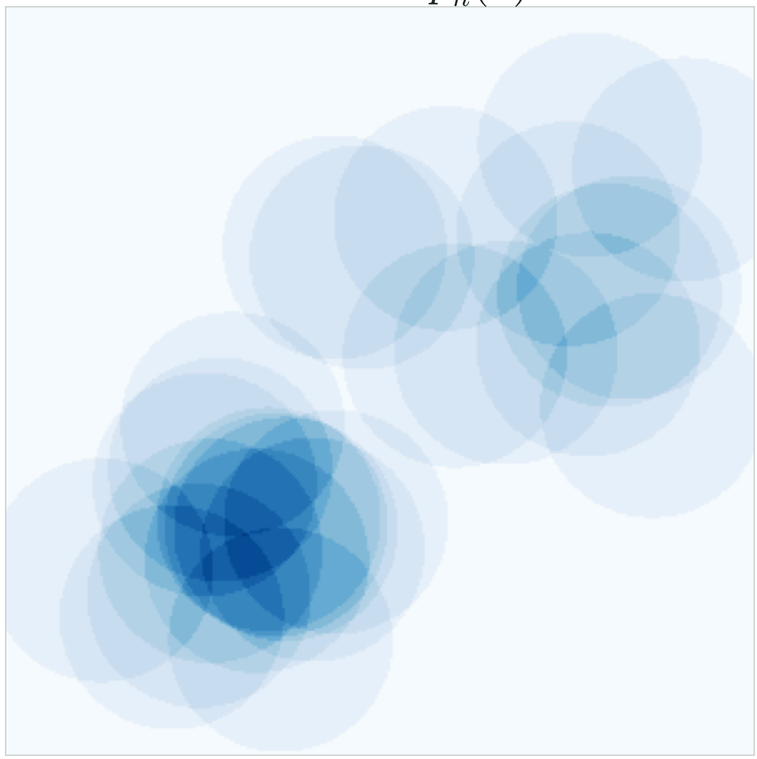
profile



samples x_i and kernels



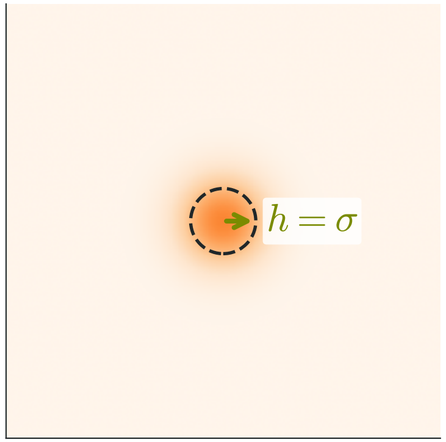
estimate $\hat{p}_h(x)$



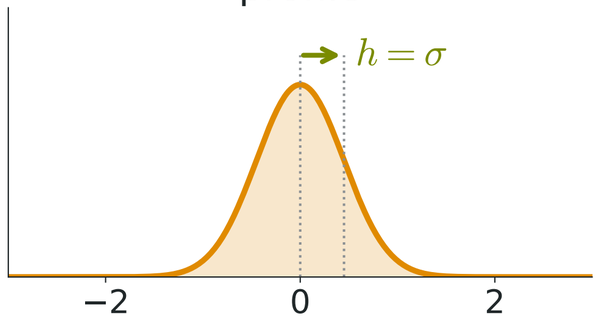
Large h: the discs overlap, but the estimate is still step-like.

Parzen windows in 2D

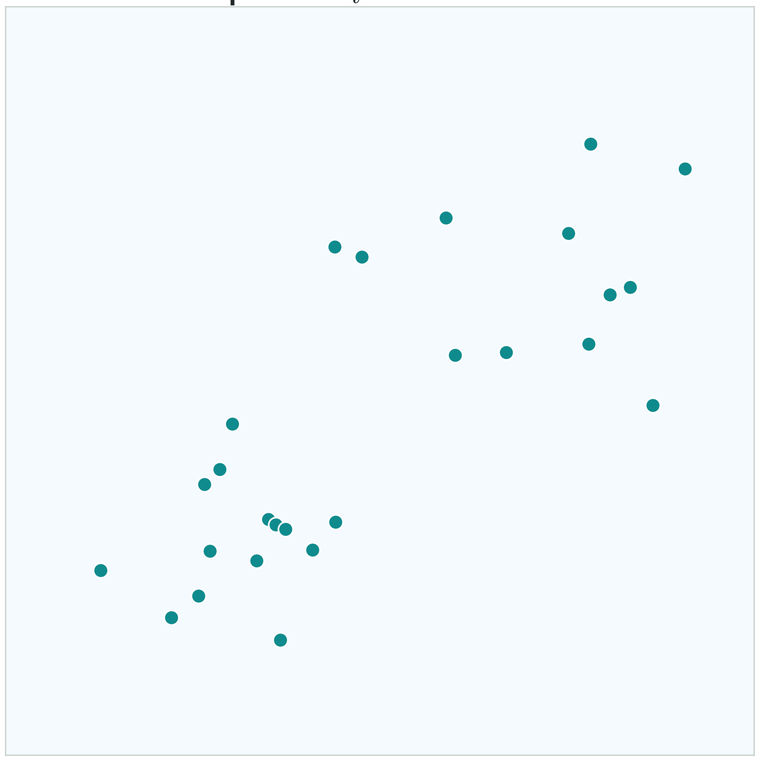
Gaussian kernel, from above



profile



samples x_i and kernels

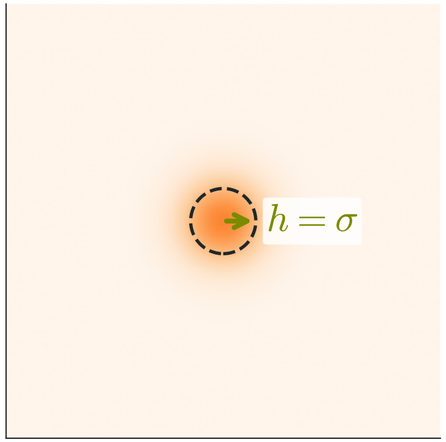


estimate $\hat{p}_h(x)$

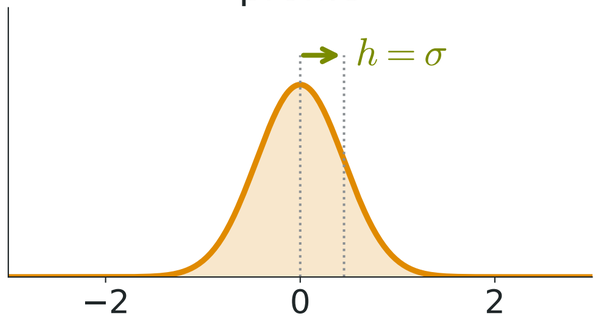
Gaussian kernel, small $h = \sigma h = \sigma$.

Parzen windows in 2D

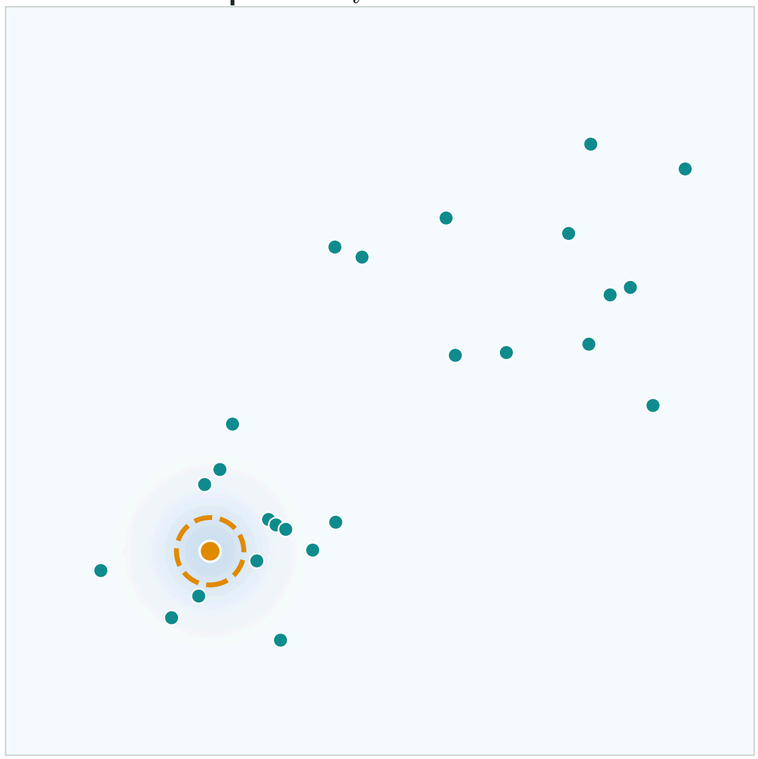
Gaussian kernel, from above



profile



samples x_i and kernels

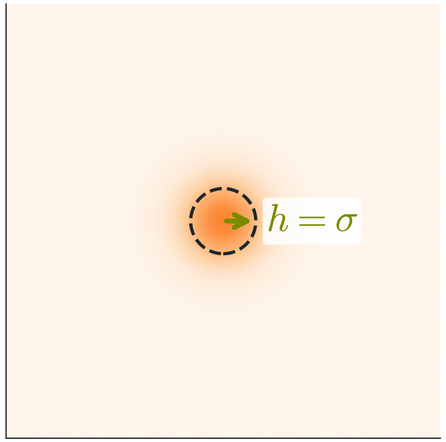


estimate $\hat{p}_h(x)$

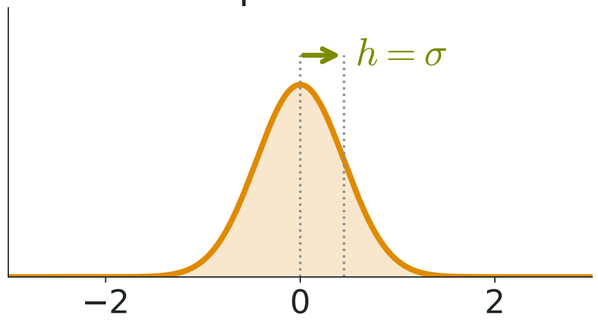
Gaussian kernel, small $h = \sigma h = \sigma$.

Parzen windows in 2D

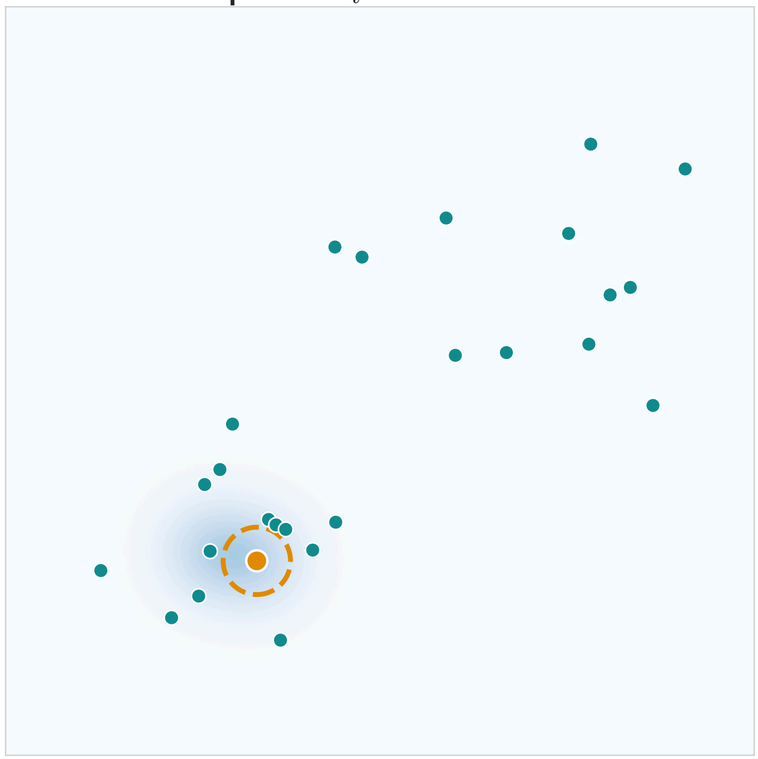
Gaussian kernel, from above



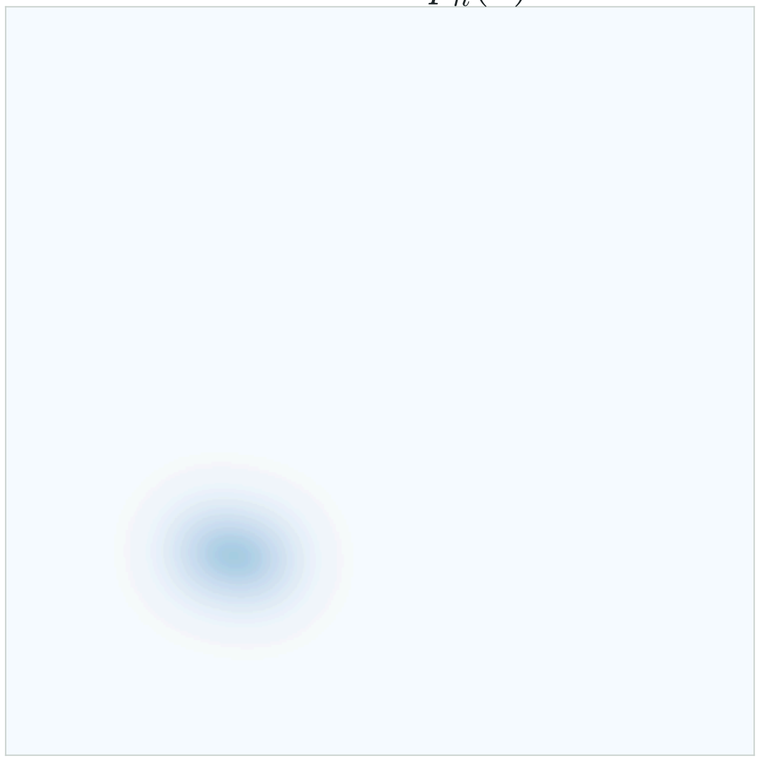
profile



samples x_i and kernels



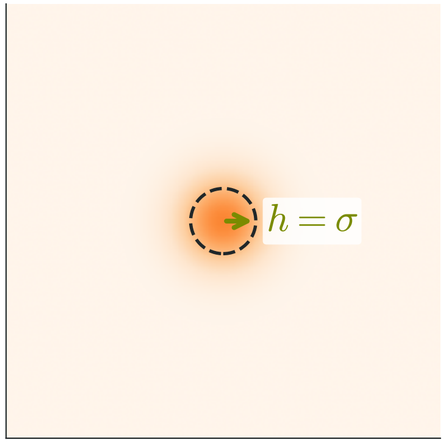
estimate $\hat{p}_h(x)$



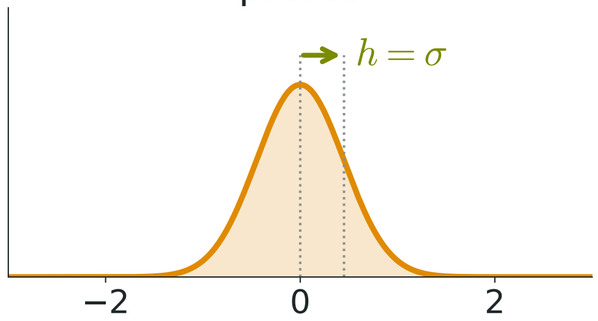
Gaussian kernel, small $h = \sigma h = \sigma$.

Parzen windows in 2D

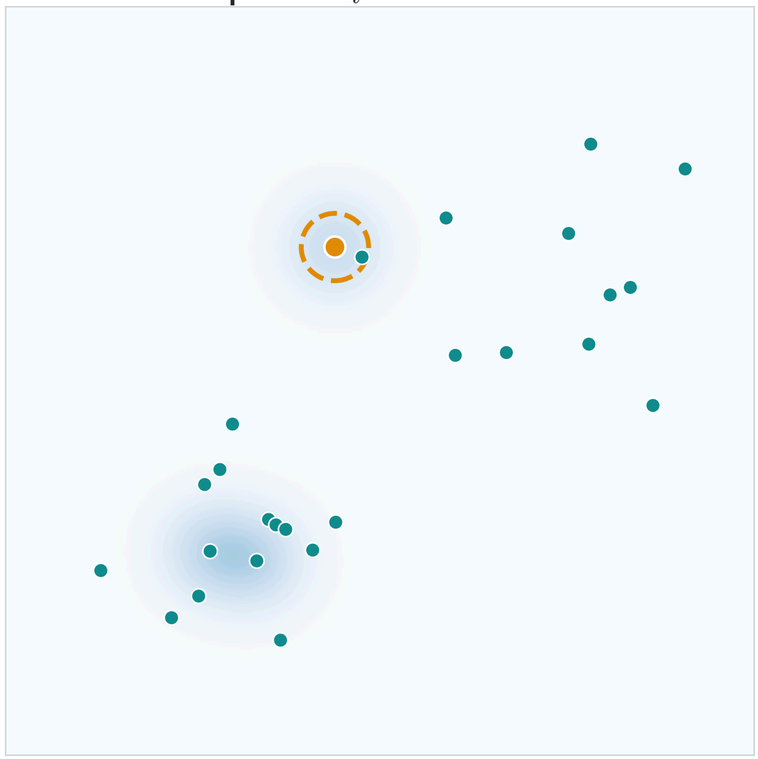
Gaussian kernel, from above



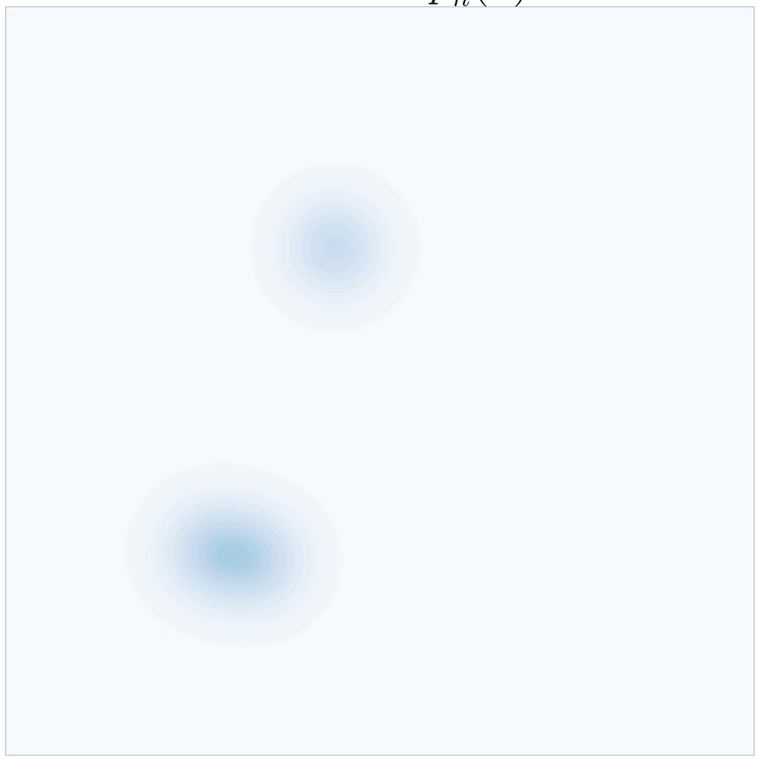
profile



samples x_i and kernels



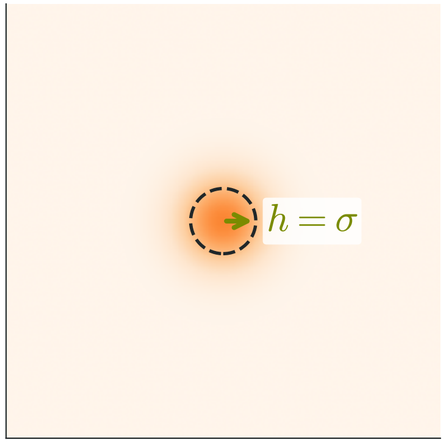
estimate $\hat{p}_h(x)$



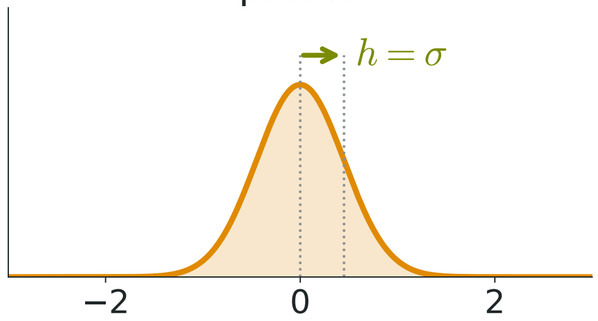
Gaussian kernel, small $h = \sigma h = \sigma$.

Parzen windows in 2D

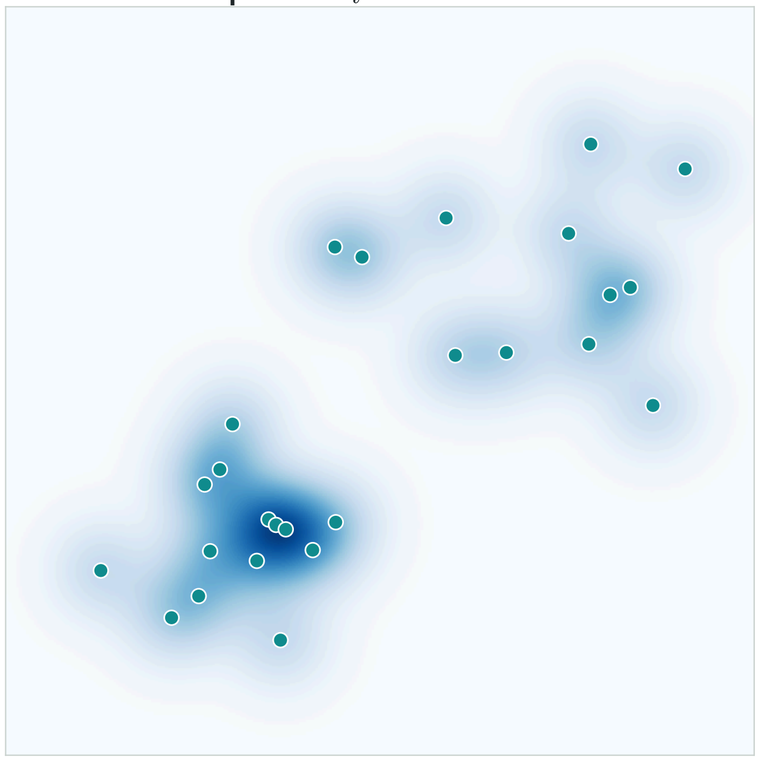
Gaussian kernel, from above



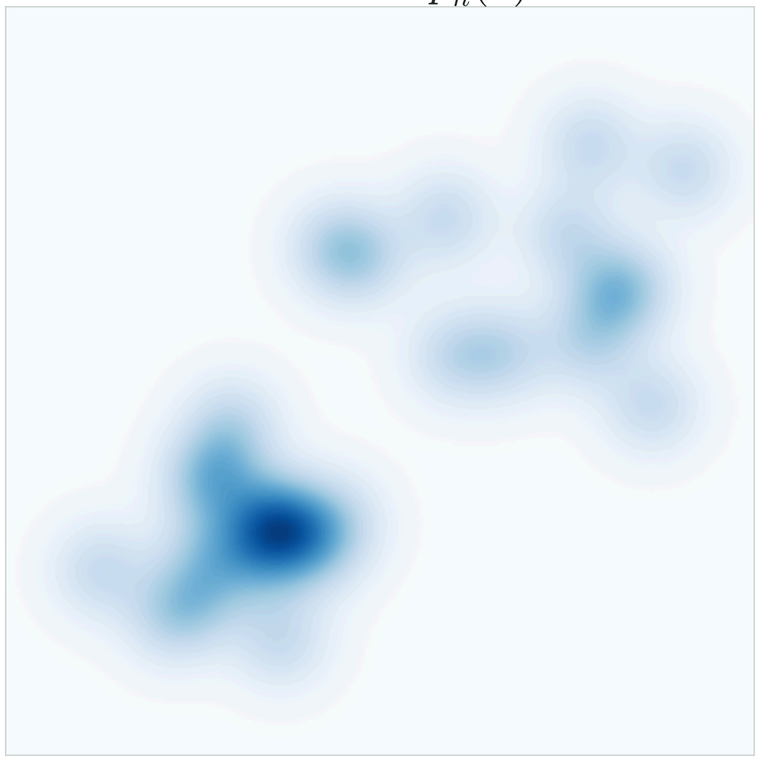
profile



samples x_i and kernels



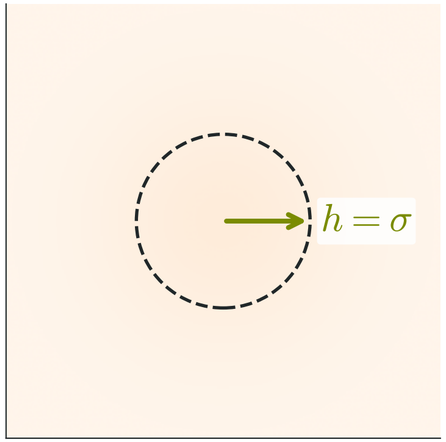
estimate $\hat{p}_h(x)$



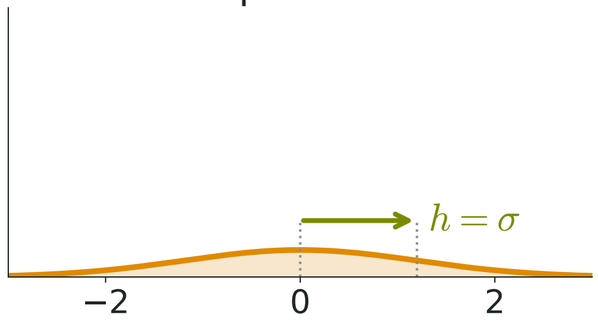
Small hh: a separate peak around every sample.

Parzen windows in 2D

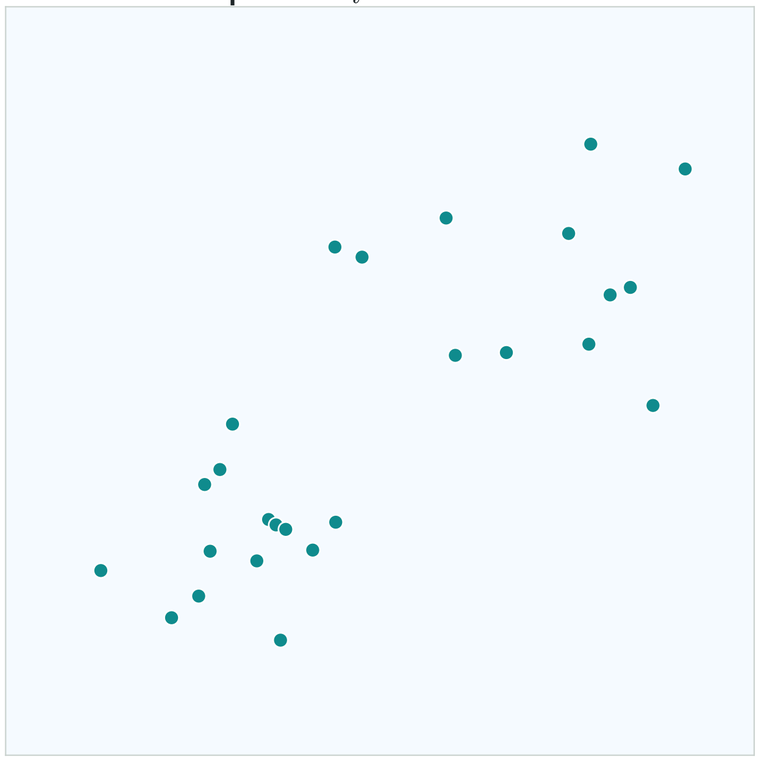
Gaussian kernel, from above



profile



samples x_i and kernels

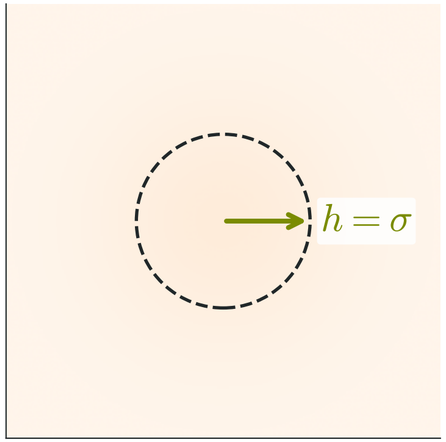


estimate $\hat{p}_h(x)$

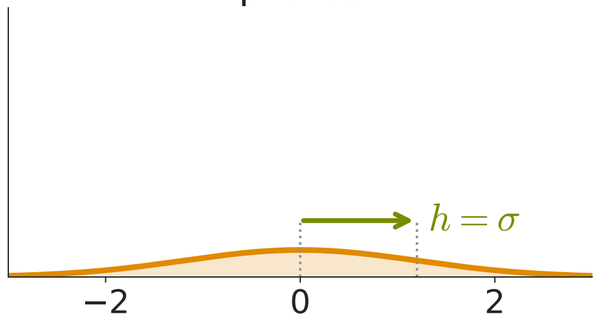
Gaussian kernel, large $h = \sigma h = \sigma$.

Parzen windows in 2D

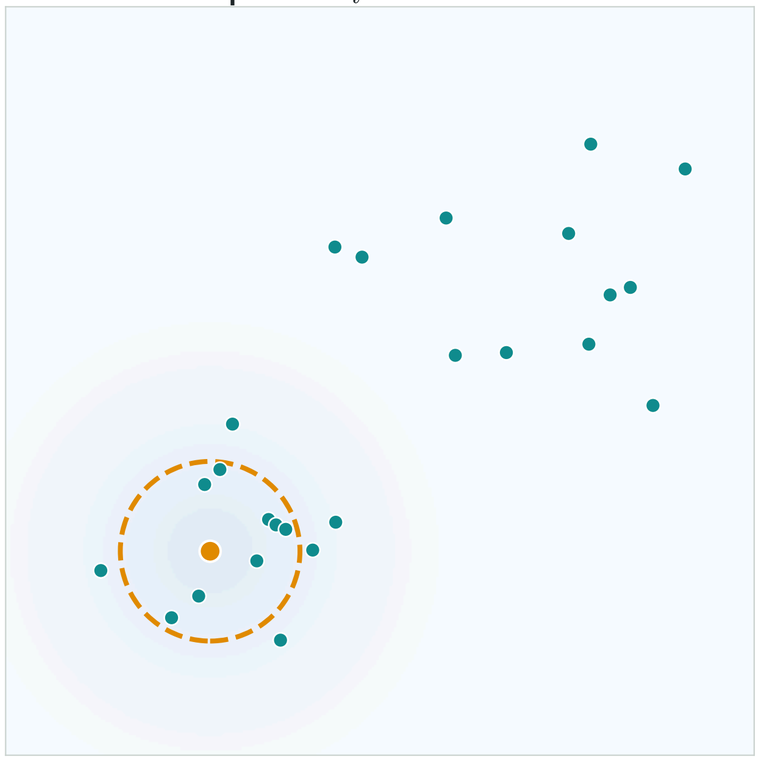
Gaussian kernel, from above



profile



samples x_i and kernels

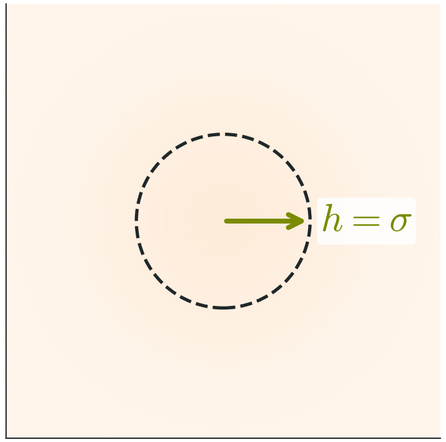


estimate $\hat{p}_h(x)$

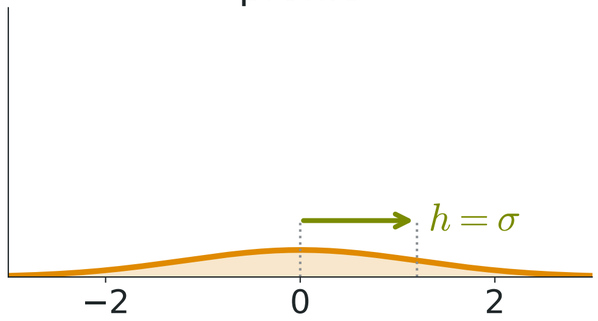
Gaussian kernel, large $h = \sigma h = \sigma$.

Parzen windows in 2D

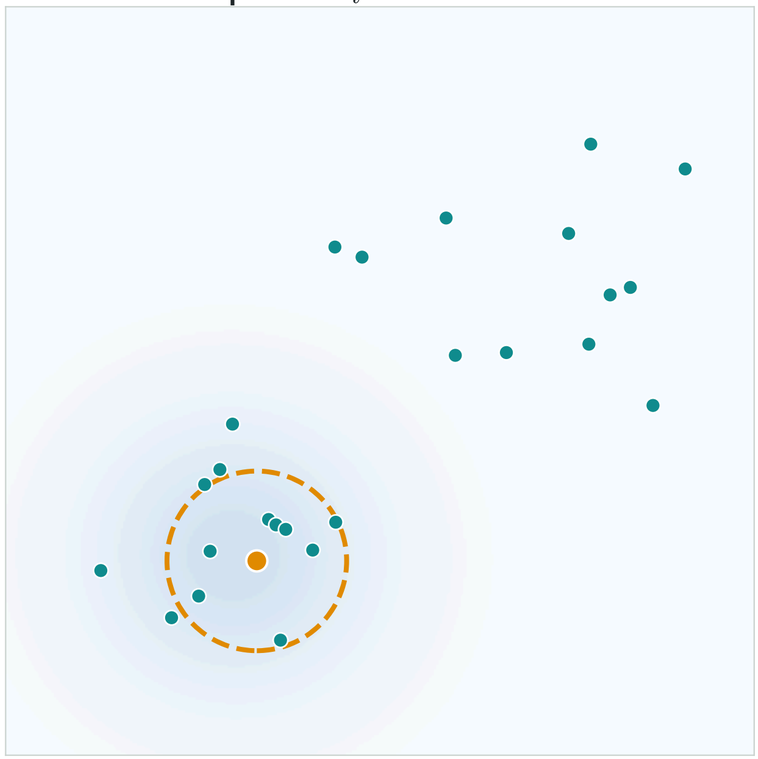
Gaussian kernel, from above



profile



samples x_i and kernels

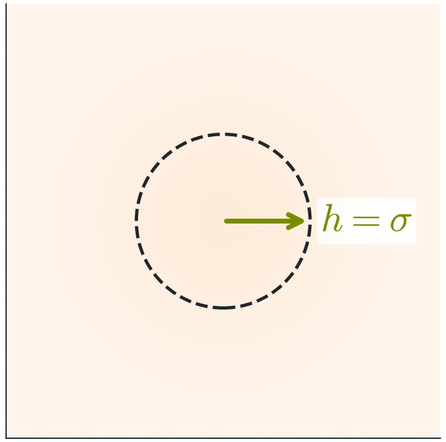


estimate $\hat{p}_h(x)$

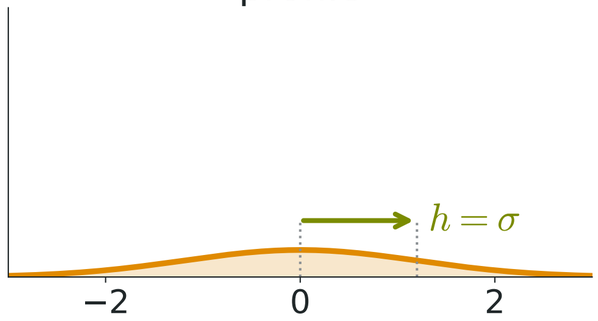
Gaussian kernel, large $h = \sigma h = \sigma$.

Parzen windows in 2D

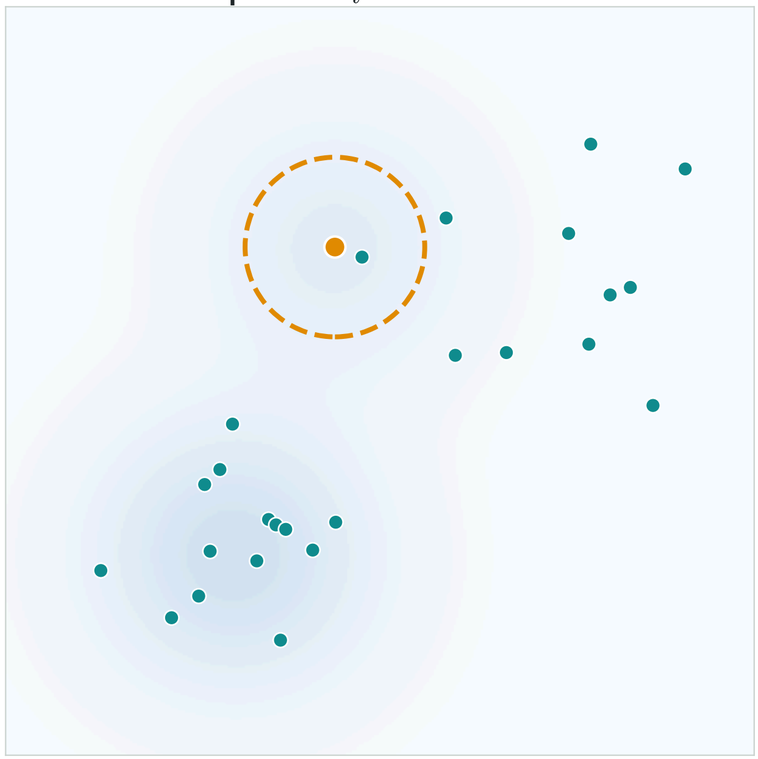
Gaussian kernel, from above



profile



samples x_i and kernels

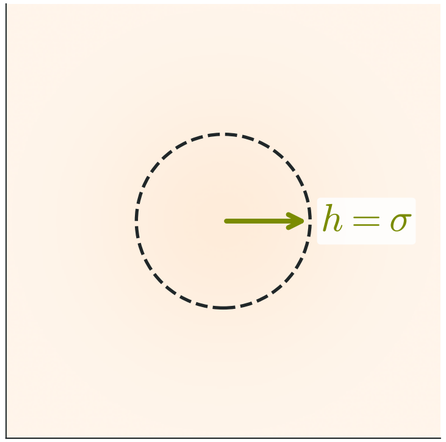


estimate $\hat{p}_h(x)$

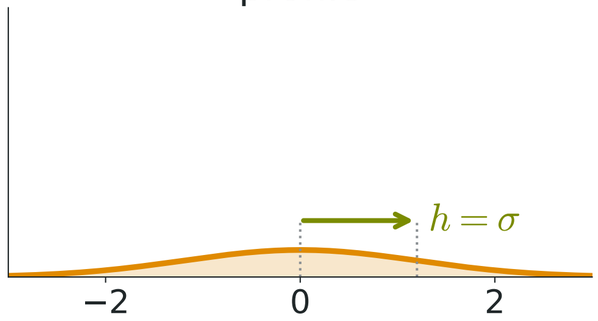
Gaussian kernel, large $h = \sigma h = \sigma$.

Parzen windows in 2D

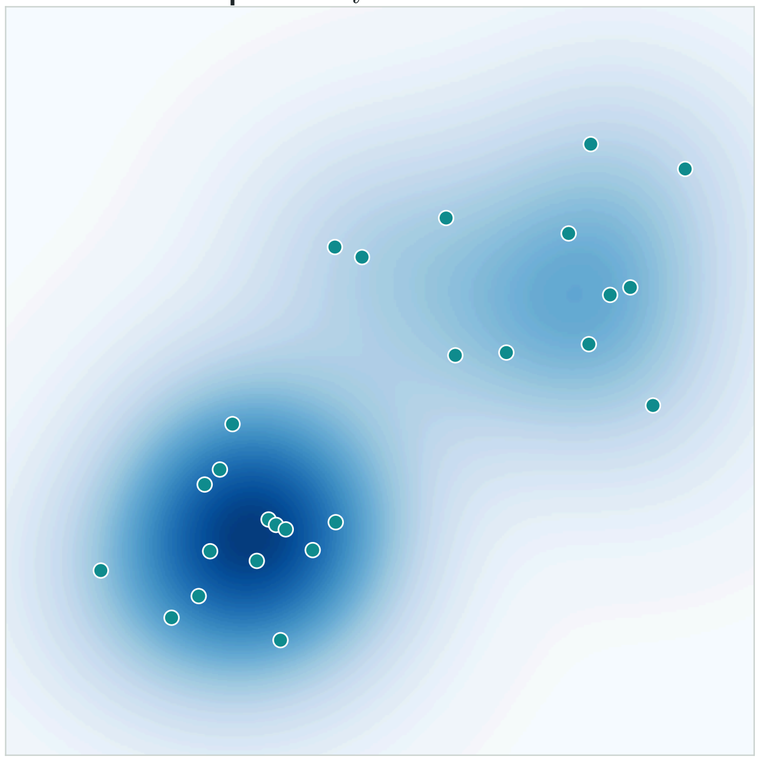
Gaussian kernel, from above



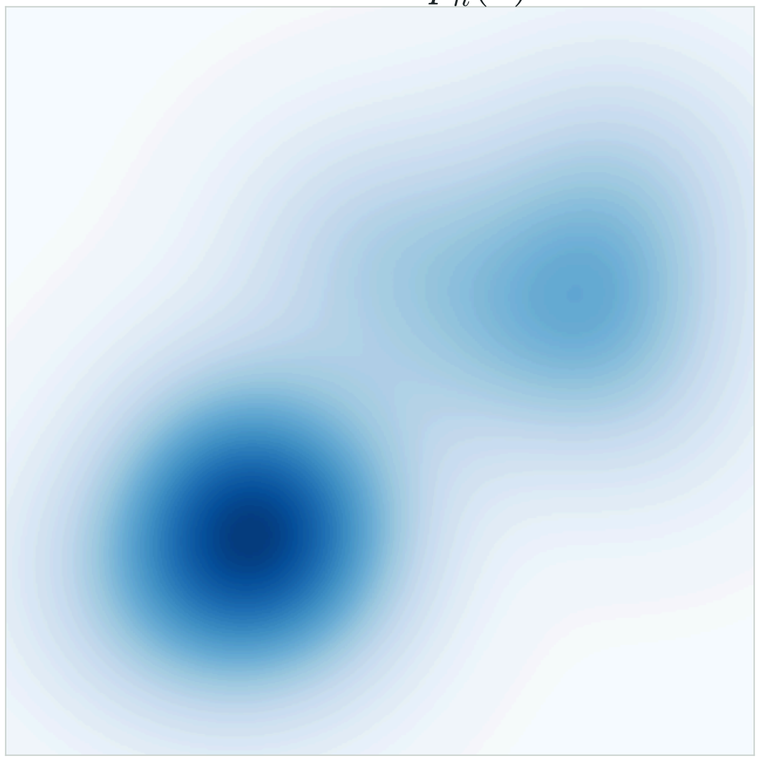
profile



samples x_i and kernels



estimate $\hat{p}_h(x)$

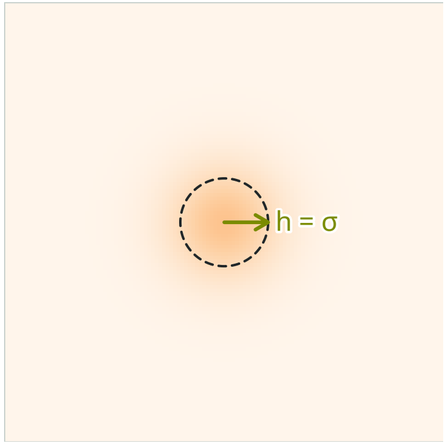


Large hh: the kernels merge into a smooth density with two clusters.

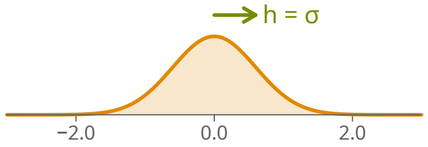
Parzen windows in 2D: try it

h = σ 0.6

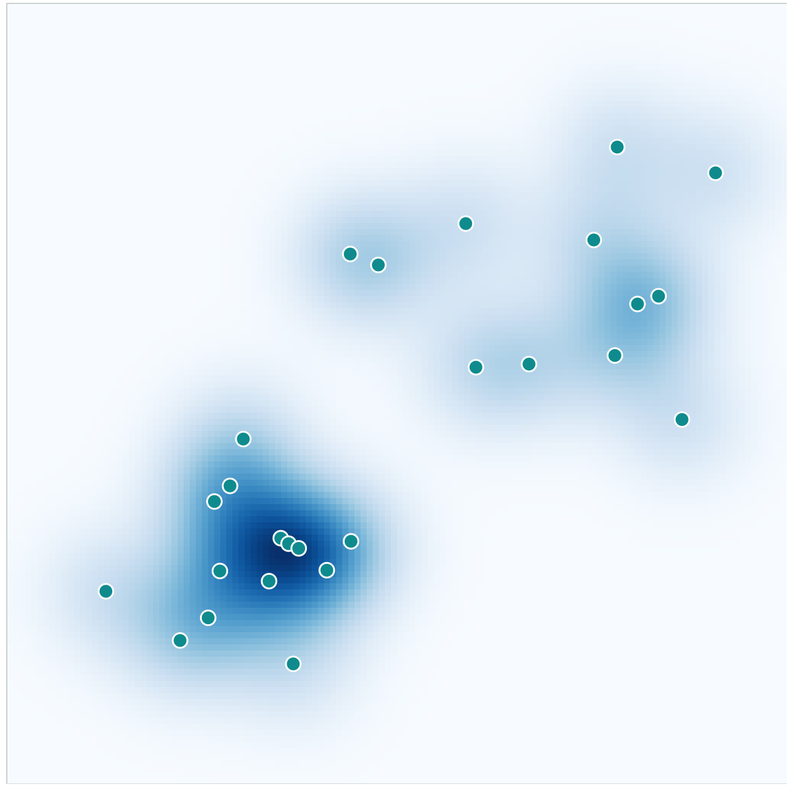
Gaussian kernel, from above



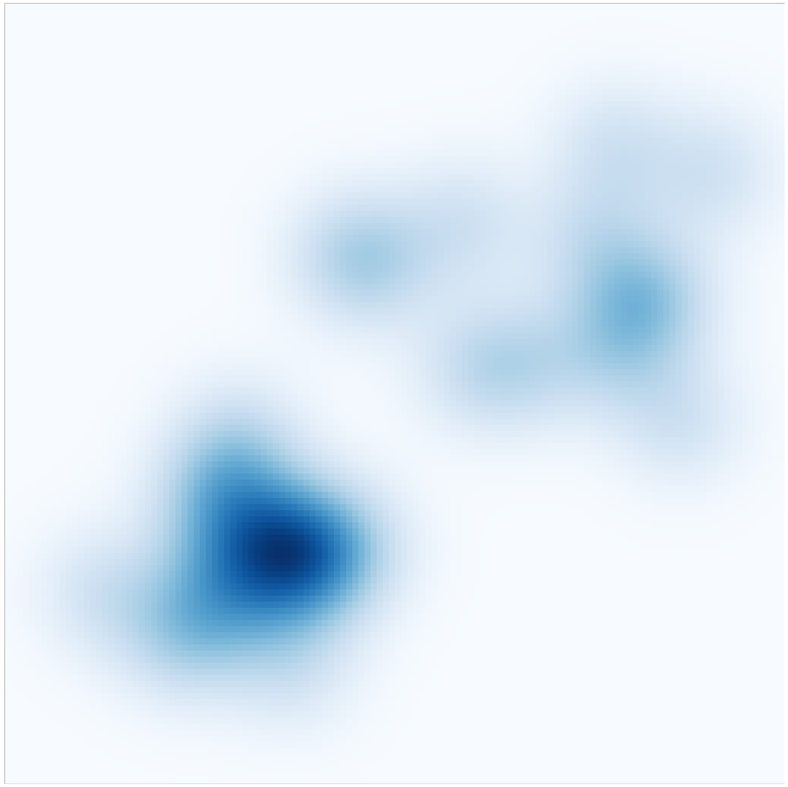
profile



samples x_i and kernels



estimate $\hat{p}_h(x)$



Parzen windows: try it

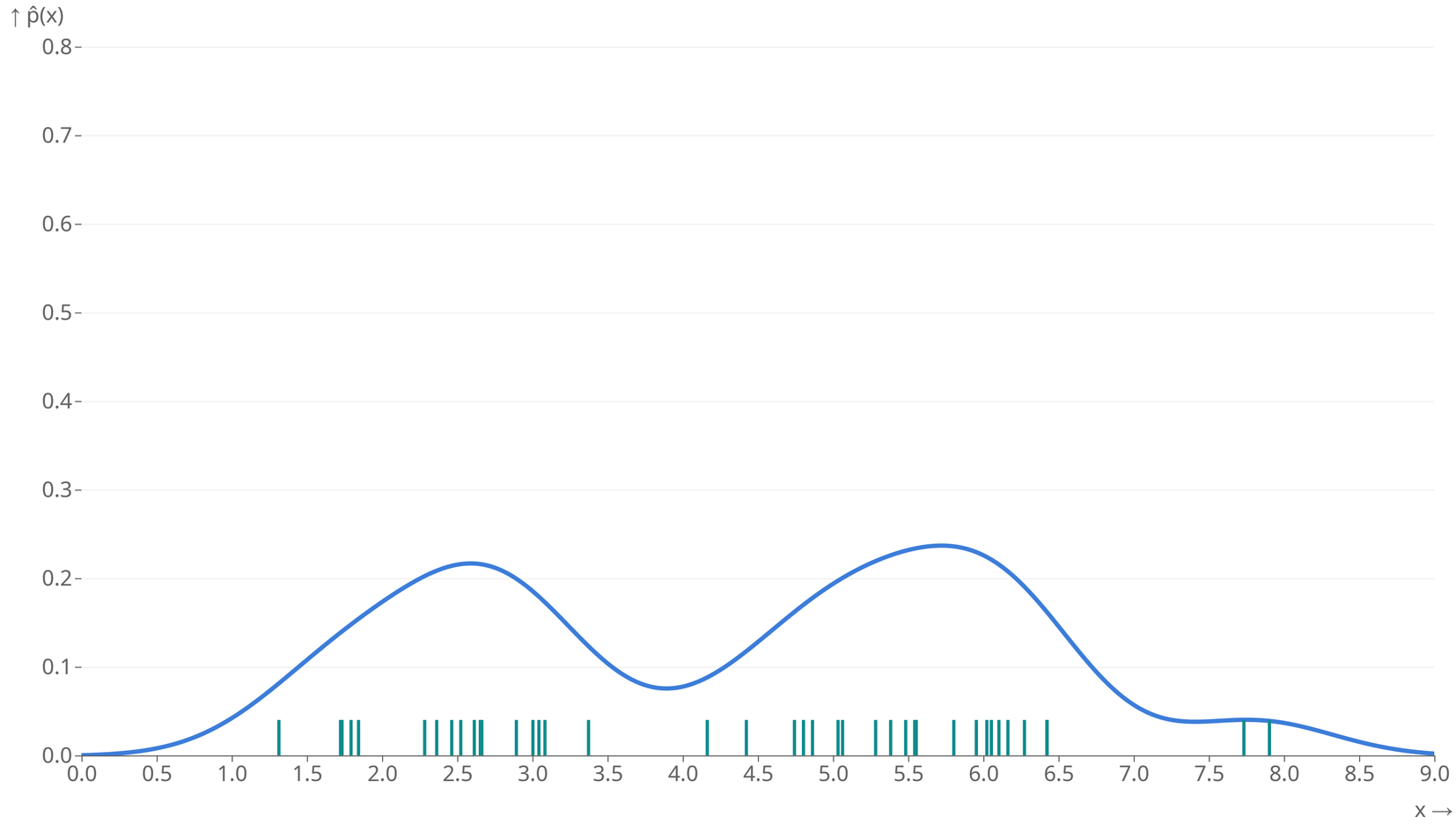
h 0.5

kernel ◉ Gaussian

○ box ○ triangle

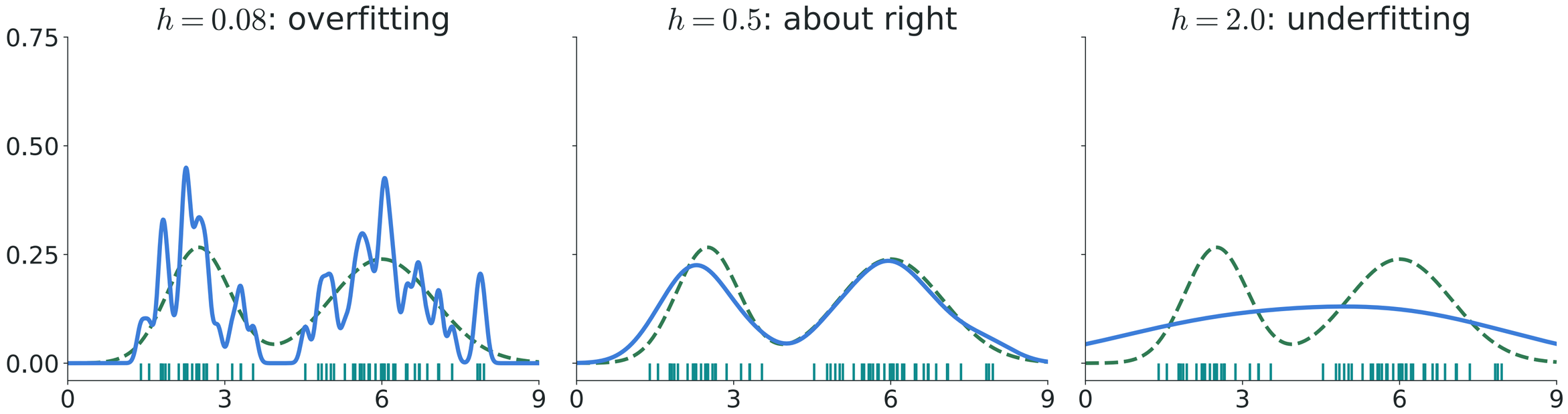
true p(x) ☐

N = **40** samples



Choosing the bandwidth h

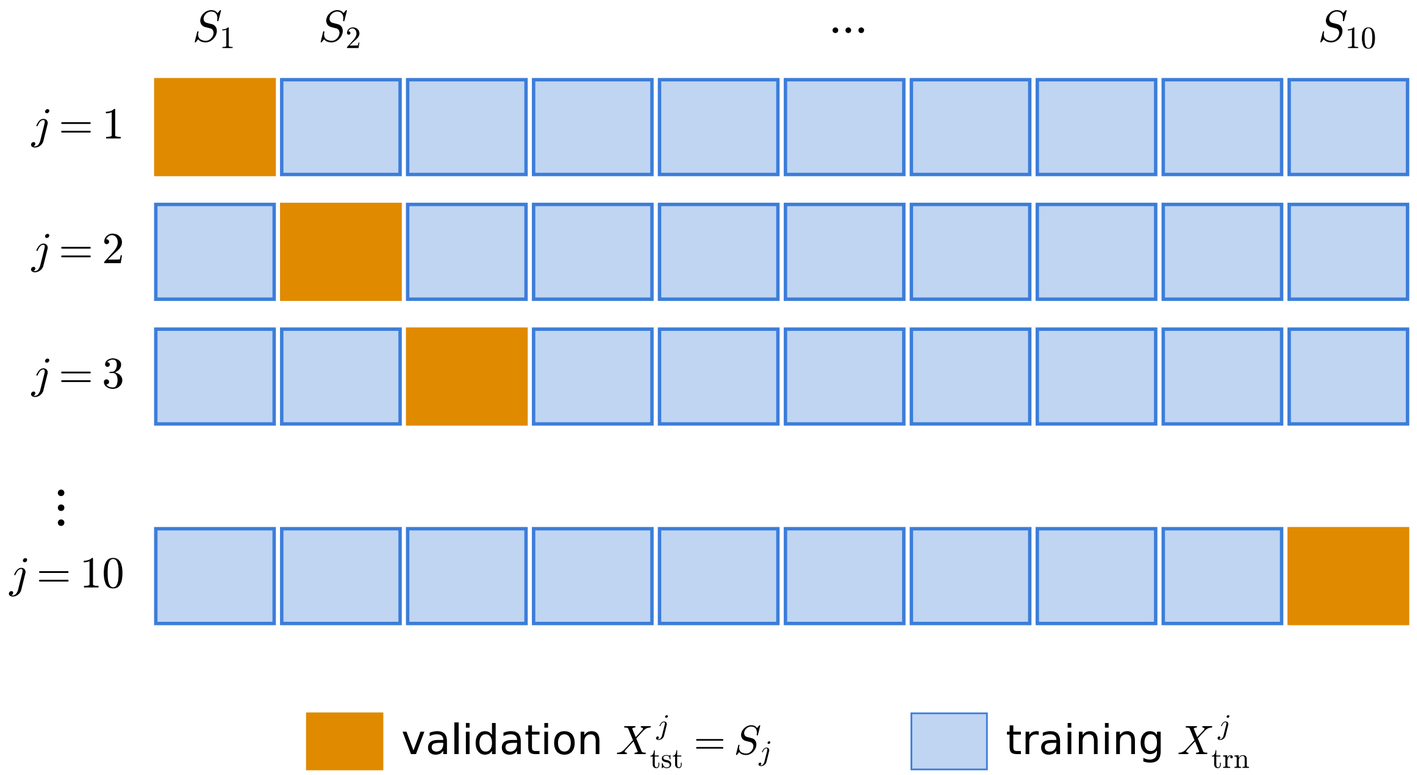
The bandwidth controls the smoothness



? Can we choose h by maximising the likelihood of the **training** samples?

! No: for $h \to 0$ every sample gets its own spike and $\hat{p}_h(x_i) \to \infty$. We need **unseen** data.

Cross-validation



$$X = \bigcup_{j=1}^{J} S_j, \quad S_i \cap S_j = \emptyset, \qquad X^j_{\text{tst}} = S_j, \quad X^j_{\text{trn}} = X \setminus S_j$$

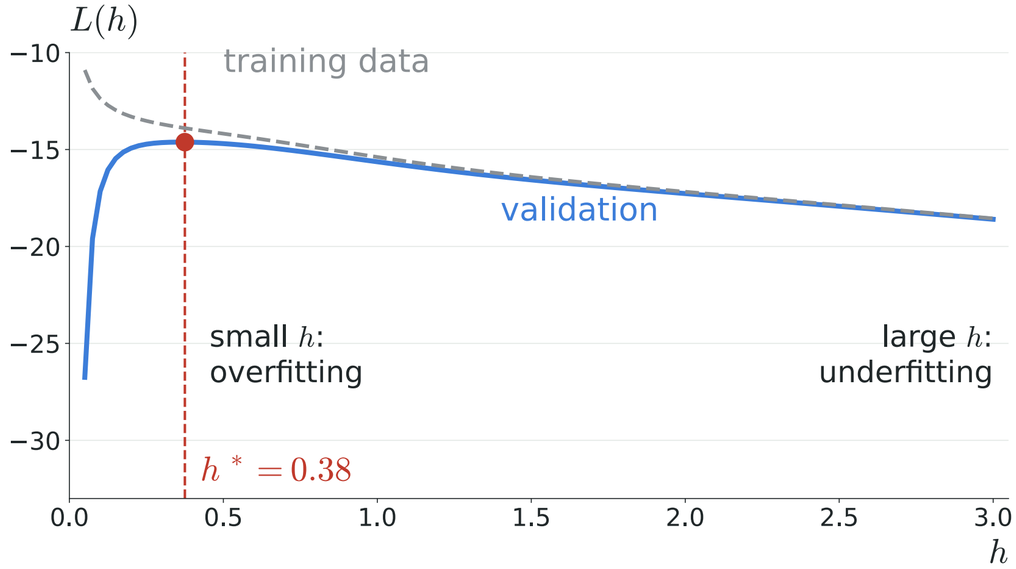
Log-likelihood on the validation set

For every split j and bandwidth h:

$$L^j(h) = \sum_{x \in X^j_{\text{tst}}} \log \hat{p}_h\left(x \,;\, X^j_{\text{trn}}\right)$$

$$L(h) = \frac{1}{J} \sum_{j=1}^{J} L^j(h)$$

$$h^* = \arg\max_h L(h)$$



? Why the logarithm?

The assignment

Assignment: Parzen windows

Part 1: estimate

`parzen(x, trn_x, h)`

Gaussian kernel, `x` can be a **vector**.

Part 2: choose h

`compute_Lh(...)`

10-fold cross-validation, `fminbound` gives h^* for each class.

Part 3: classify

`classify_bayes_parzen(...)`

Bayes with 0/1 loss, priors $p(A) = \frac{N_A}{N_A + N_C}$.

> **!** `parzen` is evaluated for **all** x and all training samples at once: build a $M \times N$ matrix of kernel values, then **average over** N.

Take-away for the lab

- [x] Without $p(x \mid k)$ we **estimate** it from samples, each class separately
- [x] In general $\hat{p}(x) = \frac{K}{NV}$: k-NN fixes K, Parzen fixes V
- [x] **Parzen:** $\hat{p}_h(x) = \frac{1}{N}\sum_i K_h(x, x_i)$, the kernel is a density
- [x] The **bandwidth** h matters more than the kernel shape
- [x] Choose h by **cross-validation**: maximise the log-likelihood of **unseen** data


"That's all Folks!"